Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Embedded Systems Engineering Modellierung

Prof. Dr.-Ing. Sebastian Schlesinger
Professor of Business Computer Science
Cybersecurity and Embedded Systems Engineering

Überblick über die gesamte Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Eingebettete Systeme

- Definition
- Eigenschaften
- Anforderungen

Entwurf Eingebetteter Systeme

Modellierungssprachen

- Modell-basierte Entwicklung
- Endliche Automaten
- Statecharts
- Timed Automata
- Signalfluss-orientierte Sprachen (Simulink)
- SystemC

Implementierung

- Programmier-sprachen
- Betriebssysteme
- Synthese

Validierung und Verifikation

- Testen
- Simulation
- Model Checking

Überblick über die gesamte Vorlesung

Eingebettete Systeme

Entwurf Eingebetteter Systeme

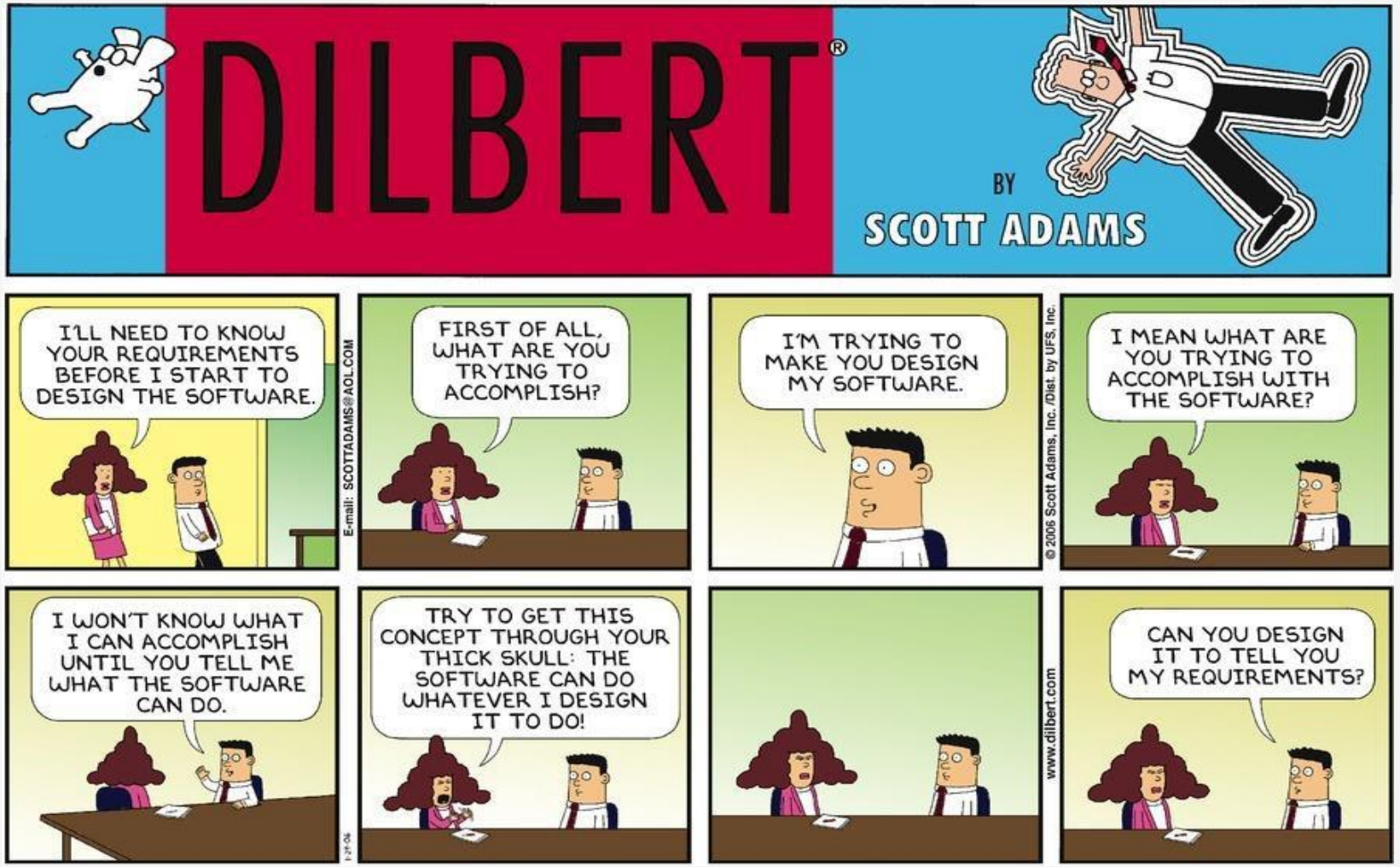
Modellierungssprachen

- Modell-basierte Entwicklung
- Endliche Automaten
- Statecharts
- Timed Automata
- Signalfluss-orientierte Sprachen (Simulink)
- SystemC

Implementierung

Validierung und Verifikation

Motivation



License covered by the Classroom Usage Statement

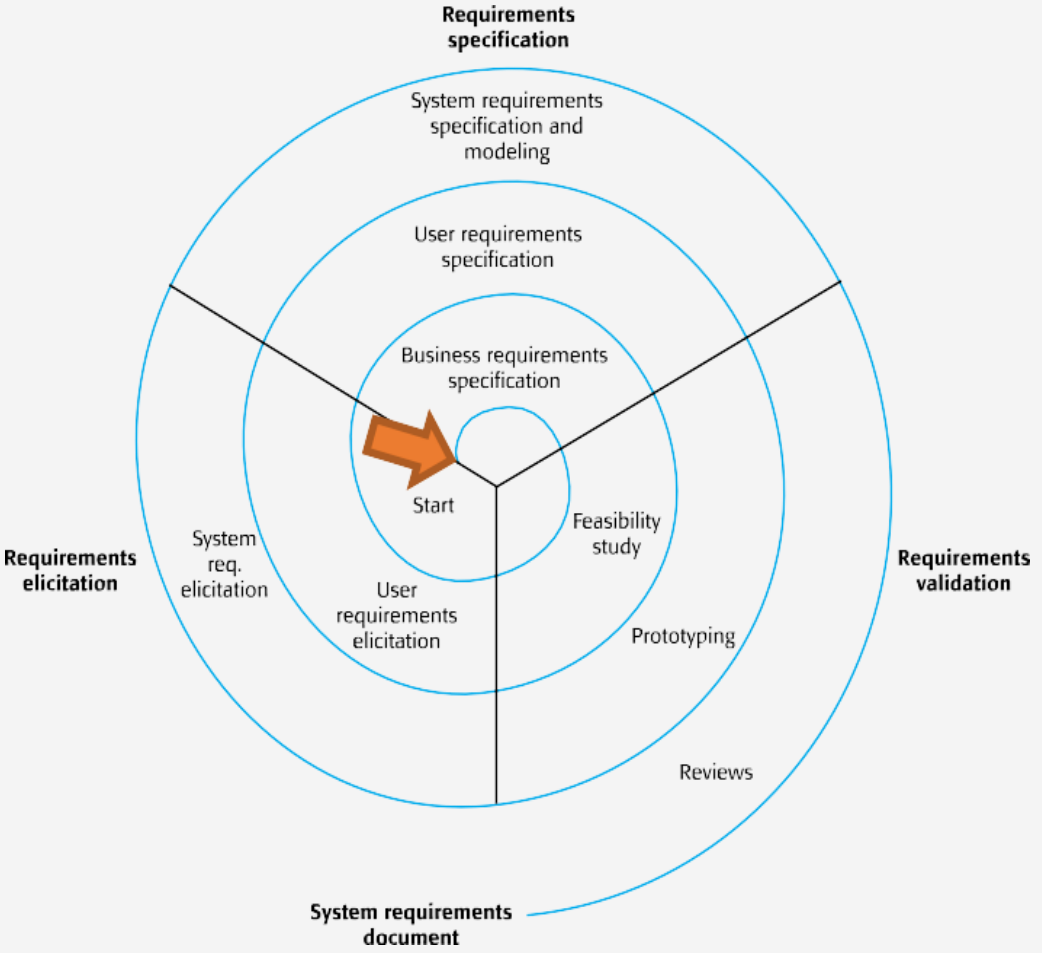
Requirements Engineering

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Umfasst typischerweise vier Elemente:

- **Ermittlung der Anforderungen**: Entwickler und Kunden bestimmen gemeinsam die Entwicklungsziele
- **Spezifikation der Anforderungen**: formale oder informale Dokumentation von Anforderungen
- **Validierung der Anforderungen**: prüfen, ob die Anforderungen das gewünschte System definieren
- **Dokumentation der Anforderungen**: Zusammenfassung der Ergebnisse im Systemanforderungsdokument

- Der tatsächliche Prozess hängt von den jeweiligen Gegebenheiten ab.
- Häufig wird dieser Prozess iterativ durchlaufen.



Ian Sommerville, Software-Engineering, Chapter 4

Normen und Regelungen

In verschieden Bereichen **müssen eingebettete Systemen bestimmte Normen einhalten**, um auf dem Markt zugelassen zu werden.

Diese Normen definieren u.a. Anforderungen an:

- Entwicklungsverfahren
- Systemarchitektur
- Qualitätssicherung
- Risikobewertung

Wichtige Normen

IEC 61508: Functional safety of electrical/electronic/ programmable electronic safety-related systems

- Allgemeiner Basisstandard zur funktionalen Sicherheit
- Veröffentlicht 1998, seit 2001 als europäische Norm EN 61508
- Führt **Safety Integrity Level (SIL)** 1-4 ein

ISO 26262: Road Vehicles – Functional Safety

- Spezialisierung von IEC 61508 für Automobilbereich seit 2011
- **Automotive Safety Integrity Level (ASIL)** A-D

(Automotive) Safety Integrity Level

Idee: **Risikoreduzierung quantifizieren**

Risiko = Schadenspotenzial × Eintrittswahrscheinlichkeit

- Beide Faktoren sollten reduziert werden, z.B. durch Redundanz

Risikoanalyse (Identifizierung + Bewertung) ermittelt **SIL-Stufe**

- **SIL 1**: niedrigste Stufe, erfordert Basis-Qualitätssicherung (QS)
- **SIL 2**: sicherheitsgetriebene QS, mehr Tests
- **SIL 3**: erschöpfende Tests, ggf. formale Methoden, Prüfung von unabhängiger Abteilung
- **SIL 4**: state-of-the-art practices, formale Methoden, Prüfung von unabhängiger Abteilung

zunehmendes Risiko

ASIL sieht Stufen A (=SIL 1) bis D (=SIL 3) vor.

Vergleich von SIL und ASIL

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Vergleich IEC EN 61508 SIL mit ISO 26262 ASIL

Safety Integrity Level

IEC EN 61508			ISO 26262		
mittlere **Wahrscheinlichkeit** eines gefahrbringenden Ausfalls bei Anforderung der Sicherheitsfunktion **PFD**	**Rate** gefahrbringender Ausfälle der Sicherheitsfunktion [Fehler/h] **PFH**	**SIL**	**ASIL**	Probabilistic Metric for random Hardware Failures **PMHF**	
-	-	QM	QM	-	-
>= 10^{-2} bis < 10^{-1}	>= 10^{-6} bis < 10^{-5} < 10.000 FIT	1	A	-	-
>= 10^{-3} bis < 10^{-2}	>= 10^{-7} bis < 10^{-6} < 1.000 FIT	2	B	< 10^{-7}	< 100 FIT
			C	< 10^{-7}	< 100 FIT
>= 10^{-4} bis < 10^{-3}	>= 10^{-8} bis < 10^{-7} < 100 FIT	3	D	< 10^{-8}	< 10 FIT
>= 10^{-5} bis < 10^{-4}	>= 10^{-9} bis < 10^{-8} < 10 FIT	4	-	-	-

http://www.uwe-lindenberg.de

1 FIT = 1 Fehler / 10^9 h

ISO 26262 **ASIL A bis ASIL D** entspricht IEC EN 61508 **SIL 1 bis SIL 3**

(A)SIL-Dekomposition

Geringere SIL durch Redundanz auf ausreichend unabhängigen Architekturelementen

Beispiel: automatische Abstandsregelung

- selbständige Bremsregelung zwischen 0-3m/s^2 (ASIL C)
- Sicherheitsanforderung (SA): keine Bremsung > 3 m/s^2 anfordern (ASIL C)
- Umsetzungen:
  - SA1: Abstandregler darf keine Bremsung > 3 m/s^2 anfordern (ASIL C)
  - SA2: Bremssteuerung darf keine Bremsung > 3 m/s^2 annehmen (ASIL C)
  - Umsetzung beider ⇒ SA1 reduziert auf ASIL A und SA2 auf ASIL B

ASIL: Beispiele

Welche ASIL-Level und Gefahren vermuten Sie bei folgenden Systemen?

- Fensterheber
- Außenlicht
- ESP (Elektronisches Stabilitätsprogramm)
- Elektronische Lenkradverriegelung

ASIL: Beispiele

- **Fensterheber**: ASIL A
  - Gefahr: Einklemmen von Fingern
- **Außenlicht**: ASIL B
  - Gefahr: Ausfall bei Nachtfahrt
- **ESP**: ASIL C
  - Gefahr: fehlerhaftes Auslösen einer Bremsung
- **Elektronische Lenkradverriegelung**: ASIL D
  - Gefahr: fehlerhafte Verriegelung während der Fahrt

Einleitung

„Computer science is a science of abstraction – creating the right model for thinking about a problem and devising the appropriate mechanizable techniques to solve it.“

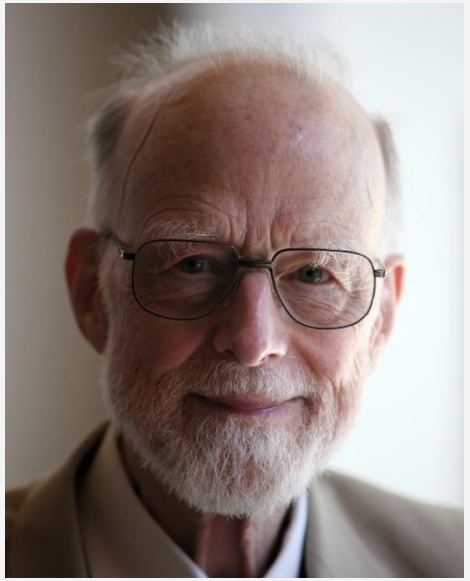
Alfred Aho, Jeffrey Ullman, 1992,
Foundations of Computer Science, Kapitel 1
frei verfügbar: infolab.stanford.edu/~ullman/focs.html

Einleitung

„Es gibt zwei Wege, ein Software-Design zu erstellen:

Entweder so einfach,
dass es offensichtlich keine Schwächen hat,
oder so kompliziert,
dass es keine offensichtlichen Schwächen hat.

Die erste Methode ist weitaus schwieriger.“



Photograph by Rama, Wikimedia Commons, Co-by-sa-2.0-fr.
https://de.wikipedia.org/wiki/Datei:Sir_Tony_Hoare_IMG_5125.jpg

Tony Hoare, 1980,
Rede zum Turing Award

Recap: wichtige Nichtfunktionale Eigenschaften eingebetteter Software

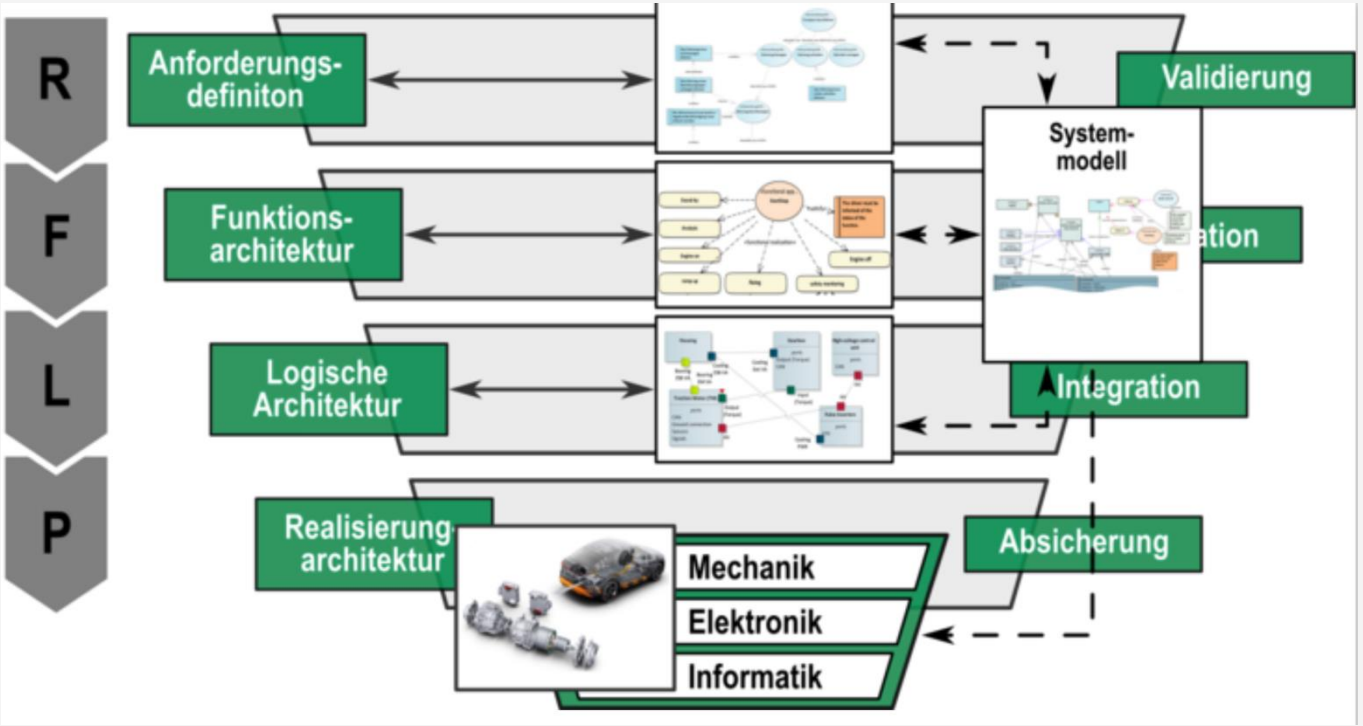
- Reaktivität
- Zeitbehaftung
- Echtzeit
- Nebenläufigkeit
- Lebendigkeit (no deadlocks)
- Heterogenität
- Sicherheit
- Zuverlässigkeit

- Viele dieser Eigenschaften sind **schwer** zu **erfassen** und zu **überprüfen**
- **Testen** ist oft zu aufwendig oder **nicht ausreichend**:
  - Testen kann nur Fehler finden, **keine Abwesenheit von Fehlern** gewährleisten
  - **Keine Obergrenze** für Rechenzeit oder Ressourcenverbrauch

Es werden **Spezifikations-, Modellierungs- und Verifikationsmethoden benötigt**, die diese Eigenschaften einbeziehen

Modellbasierte Entwicklung

- Model Driven Design / Engineering
  Idee:
  Modellierung von abstrakten Anforderungen zu immer feiner werdenden, ausführbaren, verifizierbaren Modellen bis hinunter zum Code (auch Modell)
- Modell: Abstraktion der Wirklichkeit
- Wegen hohen Anforderungen und Sicherheitskritikalität bei ES: V-Modell (agil kaum verbreitet)



Quelle: https://www.imw.tu-clausthal.de/forschung/schwerpunkte/modellbasierte-systementwicklung

Model-Driven Engineering

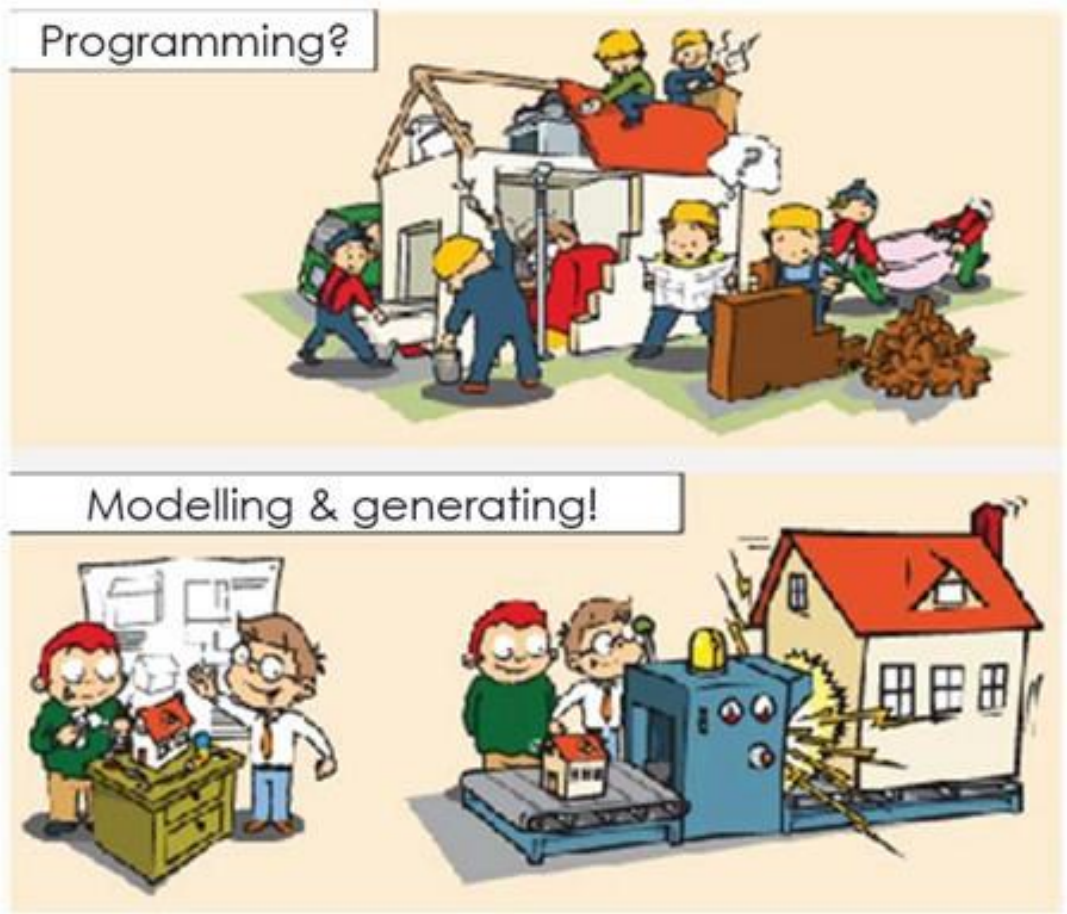
Bei der modellgetriebenen Softwareentwicklung steht die Erstellung eines abstrakten Modells im Vordergrund

- Erlaubt Entwicklung auf hohem Abstraktionsniveau
- Reduziert die Wahrscheinlichkeit von technischen Fehlern
- Erhöht die Wiederverwendbarkeit und verringert Redundanz
- Entwurfs- und Implementierungsprozess wird beschleunigt
- Die Modelle sind plattformunabhängig (frei von Ausführungsdetails)
- Domänenspezifische Modelle können direkt von Experten erstellt werden

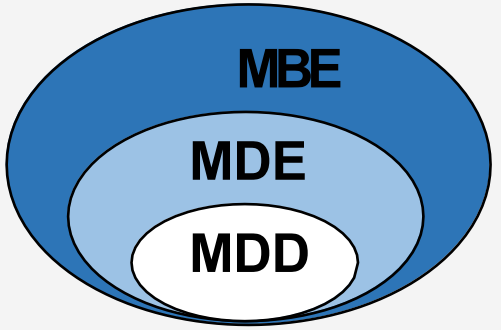
Das endgültige Produkt kann aus dem Modell abgeleitet werden

- Manuell: Durch Implementierung (Model-Based Engineering)
- Automatisch: Durch Code-Generatoren (Model-Driven Engineering)

Model-Driven Engineering



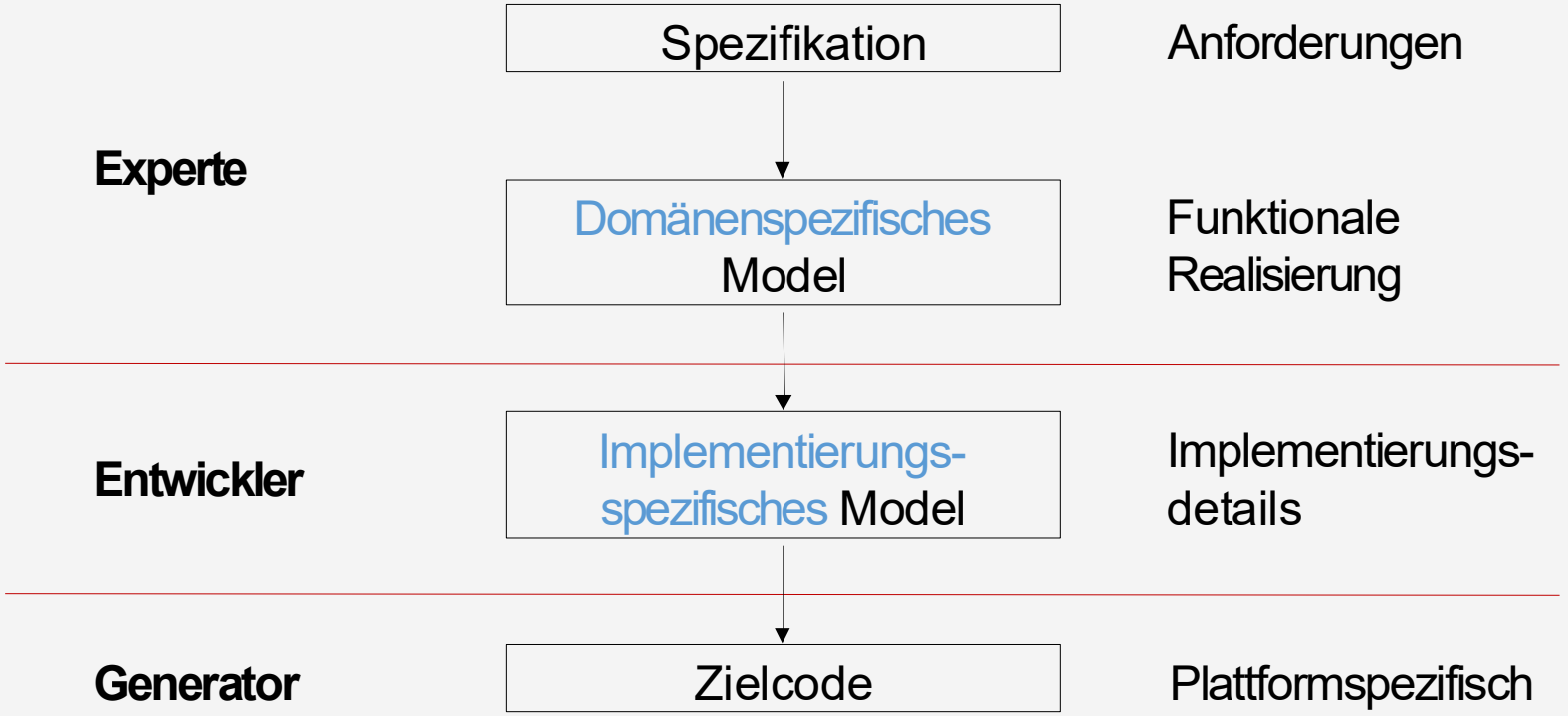
Begriffsklärung



- Model Driven Development
  - Ziel: Ableitung einer Implementierung mittels Code-Generator
- Model Driven Engineering
  - umfasst den gesamten Entwicklungsprozess
  - Z.B. auch Modellierung von Anforderungen, Model-basiertes Testen
- Model Based Engineering/ Development
  - Etwas weicher als Model Driven: Modelle sind wichtig, aber nicht „treibende“ Kraft
  - Kein generierter Code

Model-Driven Development (MDD)

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law



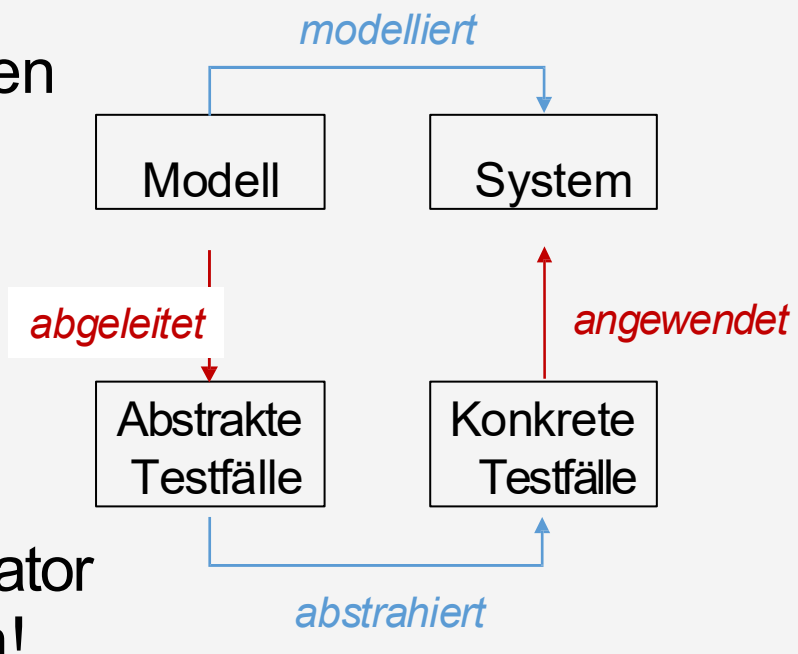
Modellbasiertes Testen

Auch **Testfälle** können direkt aus Modellen **abgeleitet** werden

- Die Modelle dienen dann als **Spezifikation** für das **System**

Erstellung und **Anwendung** von Testfällen kann automatisiert werden

- Personenunabhängige Qualität der Testfälle
- Höhere Transparenz und Wiederverwendbarkeit



Achtung: Modell und Testfallgenerator müssen validiert/verifiziert werden!

Übersicht über die heutige Vorlesung

1. Zentrale Anforderungen an Modellierungssprachen
2. Modellierung von Software
   - UML
   - Petri-Netze
3. Modellierung von Hardware
   - Blockdiagramme
   - VHDL
4. Zusammenfassung

Übersicht über die heutige Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

1. Zentrale Anforderungen an Modellierungssprachen
2. Modellierung von Software
   - UML
   - Petri-Netze
3. Modellierung von Hardware
   - Blockdiagramme
   - VHDL
4. Zusammenfassung

Spezifikation

„Spezifikation ist die (mathematisch korrekte) ***Beschreibung von Anforderungen*** *an Software und Systeme“*

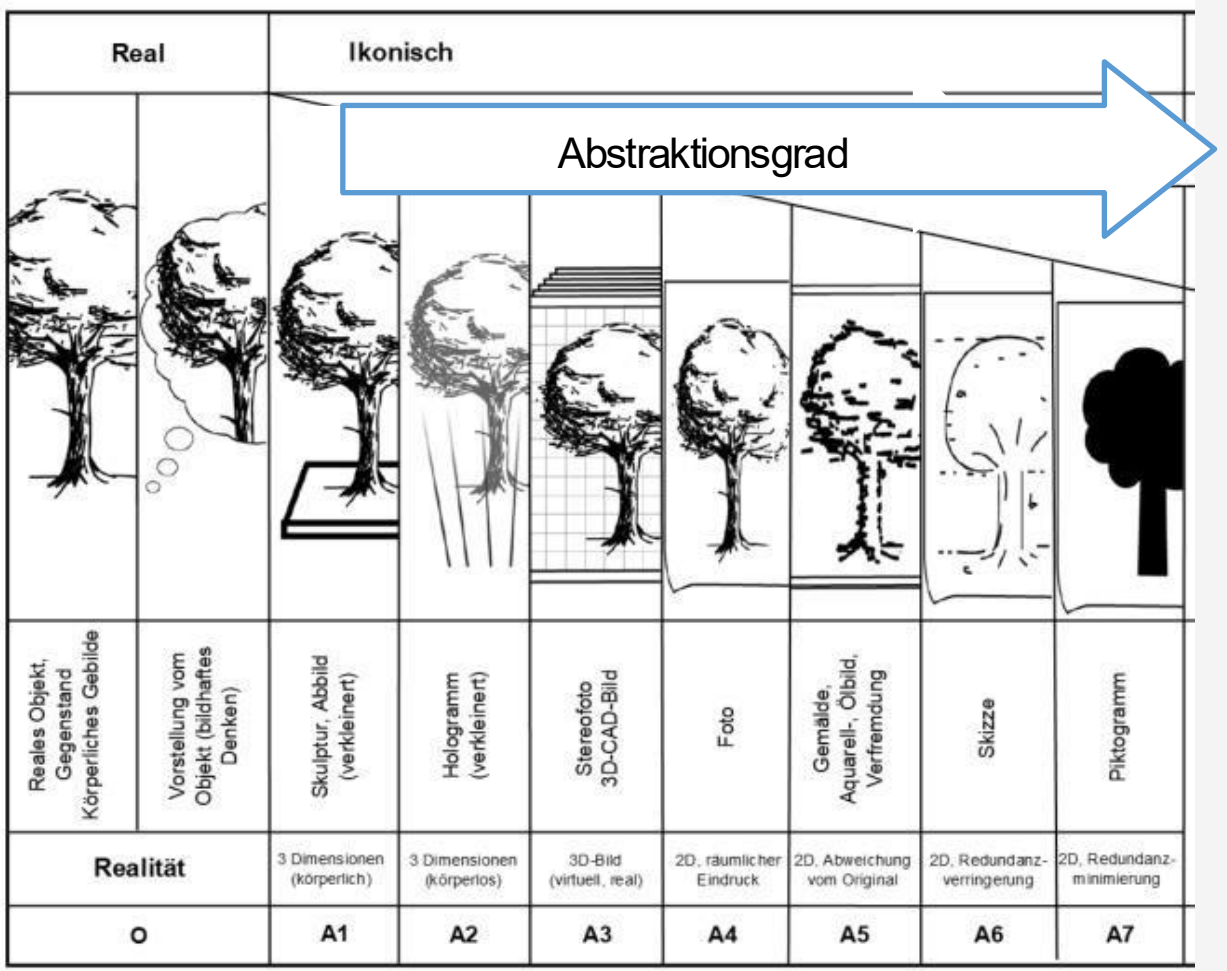
Modelle

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Ein **Modell** ist ein vereinfachtes Abbild der Wirklichkeit.

Merkmale:

- **Abbildung**: repräsentiert ein natürliches oder künstliches, konkretes oder konzeptionelles Original
- **Verkürzung**: erfasst nicht alle Attribute des Originals, sondern nur solche, die dem Modellschaffer bzw. -nutzer relevant erscheinen
- **Pragmatismus**: ersetzt das Original für bestimmte Aufgaben innerhalb bestimmter Zeitintervalle



https://kluedo.ub.uni-kl.de/frontdoor/deliver/index/docId/5288/file/Modelle+vs.+Dokumente+-+Konzeptionelle+Grundlagen+und+ihr+Einsatz+in+der+Produktentwicklung.pdf

Zentrale Anforderungen

- Für die Modellierung eingebetteter Systeme wichtig:

- Nebenläufigkeit
- Kommunikation
- Synchrones und asynchrones Verhalten
- Berücksichtigung von Zeit (und ggf. auch Energie)
- Ausführbarkeit (für Simulation und Test)
- Eindeutigkeit / Vorhersagbarkeit / Verifizierbarkeit

Prozesse

= sequentiell ausgeführte Folge von Operationen

- In der Terminologie der Betriebssystemliteratur ist dies ein Thread
- Er besitzt nicht zwangsläufig einen eigenen Adressraum wie ein Betriebssystemprozess
- Bei parallelem Rechnen werden mehrere solcher Prozesse gleichzeitig ausgeführt.
- Achtung: Abhängigkeiten zwischen Prozessen erfordern Synchronisation

Nebenläufigkeit

Zwei Vorgänge oder Prozesse A und B heißen nebenläufig, wenn sie voneinander unabhängig bearbeitet werden können.

Ungleich *Parallelität* (**gleichzeitiges** Ausführen)

- Mögliche Implementierungen: parallel, sequentiell beliebig verzahnt
- Häufig über Prozesse
- Prozessanzahl statisch oder dynamisch
- Verschachtelung möglich

Synchronisationsprobleme

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Welche Probleme können durch die nebenläufige Abarbeitung entstehen?

Synchronisationsprobleme

Welche Probleme können durch die nebenläufige Abarbeitung entstehen?

- wenn Prozesse unabhängig arbeiten, ist das gemeinsame Verhalten oft nur schlecht analysierbar und vorhersagbar (Zustandsexplosion)
- Nicht-Determinismen
  - Wer liefert zuerst ein Ergebnis?
  - *race condition*: zwei Prozesse greifen auf gemeinsame Variable zu
    ⇨kann zu verschiedenen Ergebnissen führen
  - *deadlock*: zwei Prozesse warten aufeinander
  - Verzögerungen und Konflikte bei gemeinsamen Ressourcen

Nebenläufigkeit + Nichtdeterminismus

Durch Verschachtelung sind **unterschiedliche Abläufe** möglich

Beispiel:

- Temperatur < 2°C erkannt ⇒ Spiegelheizung einschalten
- Motor ausgeschaltet ⇒ Spiegelheizung ausschalten

Temperatur < 2°C erkannt	
Spiegelheizung einschalten	
	Motor ausgeschaltet
	Spiegelheizung ausschalten

Spiegelheizung aus

Temperatur < 2°C erkannt	
	Motor ausgeschaltet
	Spiegelheizung ausschalten
Spiegelheizung einschalten	

Spiegelheizung an

Ablauf nebenläufiger Systeme

Synchroner Ablauf

alle Teilprozesse besitzen gemeinsamen Takt
Ereignisse zu definierten Zeitpunkten
➢ statische Analyse möglich (z.B. von *race conditions* und Nicht-Determinismen)

Asynchroner Ablauf

kein gemeinsamer Takt
Ereignisse sind nicht an feste Zeitpunkte gebunden
➢ sehr performant, aber nicht immer gut vorhersagbar

Bei beidem: Unverschränkte oder verschränkte Abarbeitung
Nicht zu verwechseln mit synchroner oder asynchroner Kommunikation

Nebenläufigkeit und Kommunikation

Nebenläufige Prozesse müssen miteinander kommunizieren → Prozesssynchronisation

- *shared memory*
  - Alle Prozesse können auf alle Variablen zugreifen
  - exklusiver Zugriff muss garantiert werden
    (z.B. durch Semaphoren, kritische Abschnitte und Monitore)
  - Schwer zu verstehen und zu analysieren wegen der Verschachtelung
- *message passing*
  - Prozesse kommunizieren über Nachrichten
  - synchron oder asynchron
  - blockierend oder nicht-blockierend
  - Einfach zu implementieren, aber oft langsam

Synchronität vs. Asynchronität

- Bisher (z.B. in Informatikeinführungen): Asynchrone Sprachen
  - i.d.R. sequentielle Ausführung von Befehlen mit unterschiedlicher Ausführungszeit
  - Kein globaler Takt
  - exakte Ausführungszeitpunkte und Ausführungsreihenfolge nicht vorhersagbar

- Synchrone Sprachen
  - erfordern globale Uhr/ globalen Takt
  - vermeiden Nichtderminismen
  - Ausführungszeitpunkte und Ausführungsreihenfolge vorhersagbar
  - oft weniger performant (**Warum?**)

Nebenläufigkeit + Kommunikation

Synchrone Kommunikation

Telefonieren

- Senden und Empfangen in aufeinanderfolgenden oder sogar im **gleichen Zeitschritt**
- beidseitige **Synchronisation** erforderlich

Asynchrone Kommunikation

E-Mail

- Senden und Empfangen von Nachrichten **zeitlich versetzt**
- einseitige Synchronisation
- Kommunikation über **Kanäle** (erfordert Puffer, z.B. gemeinsamen Speicher)

Verwechslungsgefahr mit Synchronen und Asynchronen Zeitmodellen!

Synchrone Kommunikation

Beispiel: regelmäßiges Abfragen von Sensordaten (*Polling*)

- Abhängig von gemessenen Werten werden Aktionen gestartet
- Danach nimmt „Messprozess“ seine Arbeit wieder auf und misst erneut Sensordaten

➔ Synchrone Kommunikation zwischen Messprozess und Prozess, der darauf reagiert

Asynchrone Kommunikation

Senden und Empfangen von Nachrichten kann **zeitlich versetzt** geschehen

Problem: Nachrichten müssen **zwischengelagert** werden

- **gleichzeitiger Zugriff** muss **verhindert** werden

Lösung mit **Semaphore** (oder Monitoren und Locks)

- z.B. in Betriebssystemen, signalisiert Belegung von Betriebsmitteln

Lösung mit **gemeinsamem Speicher**

- z.B. durch Reservierung eines Teils des Hauptspeichers
- auch hier müssen Prozesse benachrichtigt werden, dass Botschaften für sie vorliegen

Semaphore

Begriff stammt **aus der Eisenbahntechnik**

Signal zeigt an, ob ein Zug eine **„geschützte Ressource“ (Gleisabschnitt)** belegen darf

Von Dijkstra 1965 erstmals **auf Informatik** angewendet zum **Schutz von Speicherbereichen**

Zwei Operationen:

- Belegen
- Freigeben

Ablauf mit Semaphore

- Beim **Betreten eines kritischen Abschnitts** rufen Prozesse die Operation **Belegen** auf
- Beim **Verlassen des kritischen Abschnitts** rufen Prozesse die Operation **Freigeben** auf
- wenn die Semaphore **bereits belegt** ist werden Prozesse, die die Operation **Belegen** **aufrufen**, **blockiert**
- wenn der erste Prozess die **Semaphore freigegeben** hat, kann der **nächste Prozess** den kritischen Abschnitt betreten

Berücksichtigung von Zeit

Zeit ist immer wichtig, denn Umgebungsreaktionen geschehen in Echtzeit.

- Messen von Zeit (*timer/clocks*)
- Verzögern von Prozessen (*delay*)
- Modellierung von Timeouts
- Modellierung von deadlines und schedules

Ausführbarkeit

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

- Ausführbare Modelle ermöglichen Simulation und Testen
- Simulation
  - Beobachtung des Verhaltens eines simulierten Systems / Modells
  - Ausführung mit ausgewählten Eingaben
  - erfordert ausführbare Modelle
- Testen
  - Beobachtung des Verhaltens eines realen Systems / der Implementierung
  - Ausführung mit ausgewählten Eingaben
  - erfordert in eingebetteten Systemen häufig ebenfalls Modelle
- ⇨Hardware in the Loop (HiL), Software in the Loop (SiL), Umgebungsmodelle

Eindeutigkeit / Verifizierbarkeit

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

- Semantisch eindeutig: es gibt eine mathematisch präzise Definition des Effekts, den ein Ausdruck oder ein Modellierungskonstrukt auf das Systemverhalten hat
- Natürliche Sprache ist mehrdeutig:

> „alle 30 Sekunden werden die Werte der Sensoren abgelesen; wenn die Standardabweichung 0,25 überschreitet, wird die Normalisierungsprozedur ausgeführt werden, anschließend werden die Werte an das Analysepaket weitergegeben werden."

- Verifikation
  - Überprüfung der Korrektheit von Systemen
  - für alle möglichen Eingaben
  - Voraussetzung: eindeutige (formale) Definition von Anforderungen und System

Lebendigkeit & Fairness

- Lebendigkeit
  - Eingebettete Systeme terminieren meist nicht oder nur im Fehlerfall
  - Verbotene Situation: Übergang in einen Zustand, in dem das System keine weiteren Ereignisse akzeptiert (Deadlock)
  - Auch nicht erlaubt: Übergang in einen Zustand, in dem das System zwar noch auf einige, aber nicht mehr alle möglichen Ereignisse reagiert (Livelock)

- Fairness
  - Idee: Jeder Prozess kommt an die Reihe
  - „No starvation“
  - Jede zur Ausführung bereite Komponente wird auch irgendwann ausgeführt

Modellierungssprachen

Einige Modellierungssprachen sind eventuell bereits aus dem Studium bekannt:

UML Petri-Netze

Automaten VHDL Statecharts Blockdiagramme

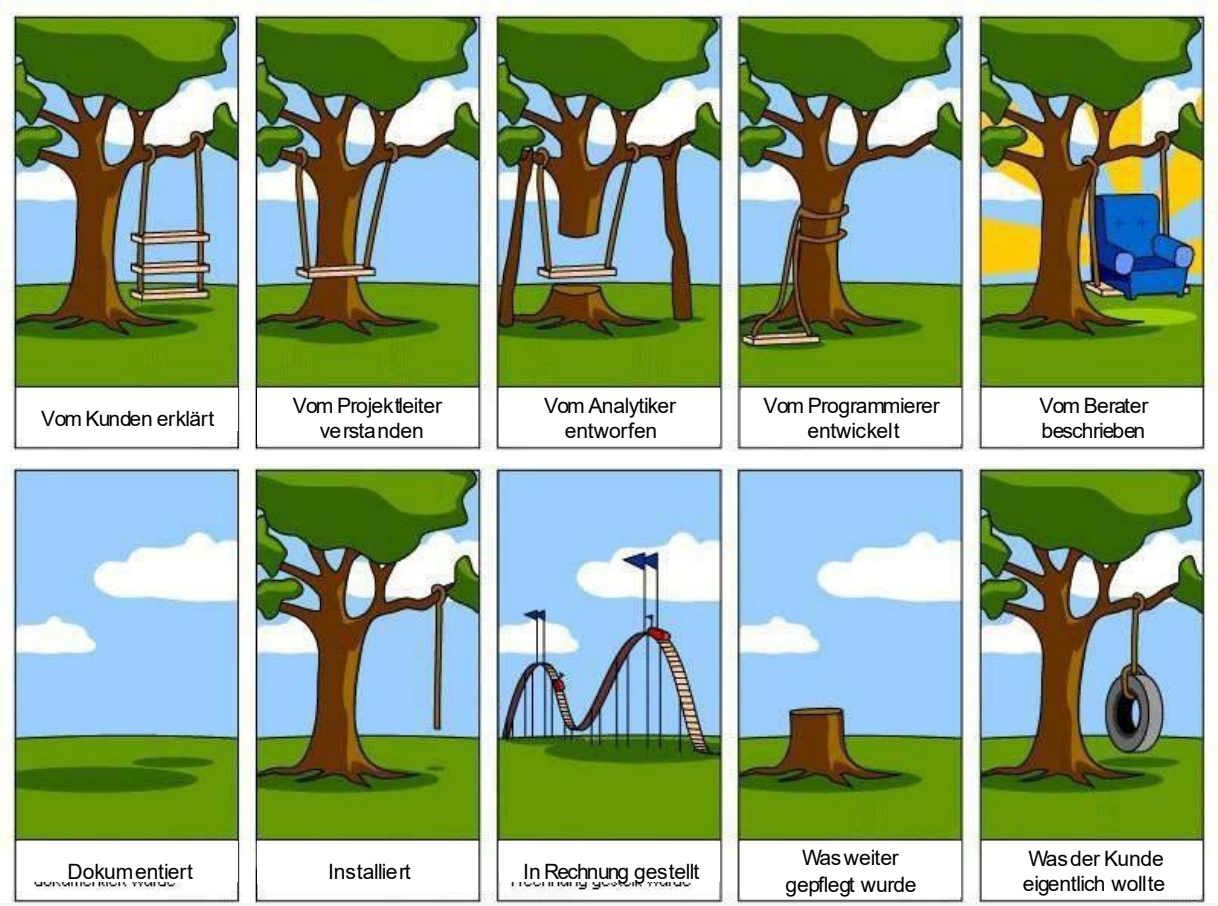
- Jede Modellierungssprache hat Stärken und Schwächen
- Nicht alle sind gleich gut geeignet für eingebettete Systeme

Übersicht über die heutige Vorlesung

1. Zentrale Anforderungen an Modellierungssprachen
2. Modellierung von Software
   - UML
   - Petri-Netze
3. Modellierung von Hardware
   - Blockdiagramme
   - VHDL
4. Zusammenfassung

Textuelle Anforderungsspezifikation

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law



Textuelle Anforderungsspezifikation

Zwei verbreitete Varianten:

Spezifikation in natürliche Sprache:

- *Ausführliche* textuelle Beschreibung der Anforderungen an ein System bzw. einer Funktionalität
- **Problem:** Häufig entsteht viel Interpretationsspielraum, da natürliche Sprache nicht eindeutig ist

Strukturierte Spezifikation:

- Übersichtlichere Erfassung von Anforderungen im Vergleich zu natürlicher Sprache
- Tabellarische Erfassung von Anforderungen mit einheitlichen Eckdaten.

Beispiel:

Funktion	Absicht	Überwachte Variablen	Geregelte Variablen	Pre-Condition	Post-Condition	Verlauf	Regeln	Sonderfälle

Modellierungssprachen

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

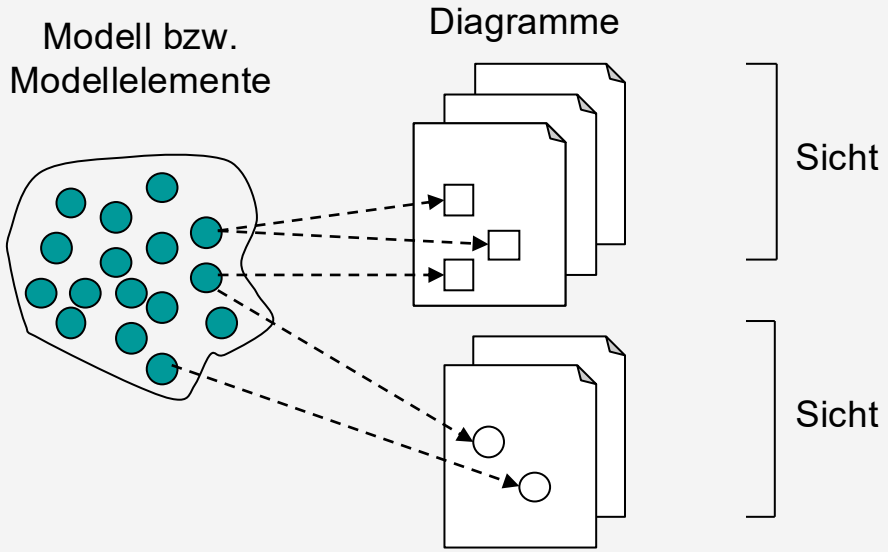
Modellierungs-sprache	Software	Hardware	Diskretes Verhalten	Kontinuierliches Verhalten	Nebenläufigkeit	Kommunikation			Zeitbehaftung		Simulierung und Verifikation
						Message Passing	Geteilte Variablen	Signal-orientiert	Synchron	Asynchron	
VHDL	✗	✓	✓	✗	✓	✗	✗	✓	✓	✗	✓
UML	✓	✗	✓	✗	✓	✓	✓	✗	✗	(✓)	(✓)
FSM	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓
Petri-Netze	✓	✗	✓	✗	✓	✓	✗	✗	✗	(✓)	✓
Statecharts	✓	✓	✓	✗	✓	✓	✓	✗	✗	(✓)	✓
Safe State Machines	✓	✓	✓	✗	✓	✓	✗	✓	✓	✗	✓
Timed Automata	✓	✓	✓	(✓)	✓	✓	(✓)	✗	✗	✓	✓
Simulink	✓	✓	✓	✓	✓	✗	✗	✓	✓	✗	✓
SystemC	✓	✓	✓	✗	✓	✓	✗	✗	✓	✗	✓

1: mit Zeiterweiterung

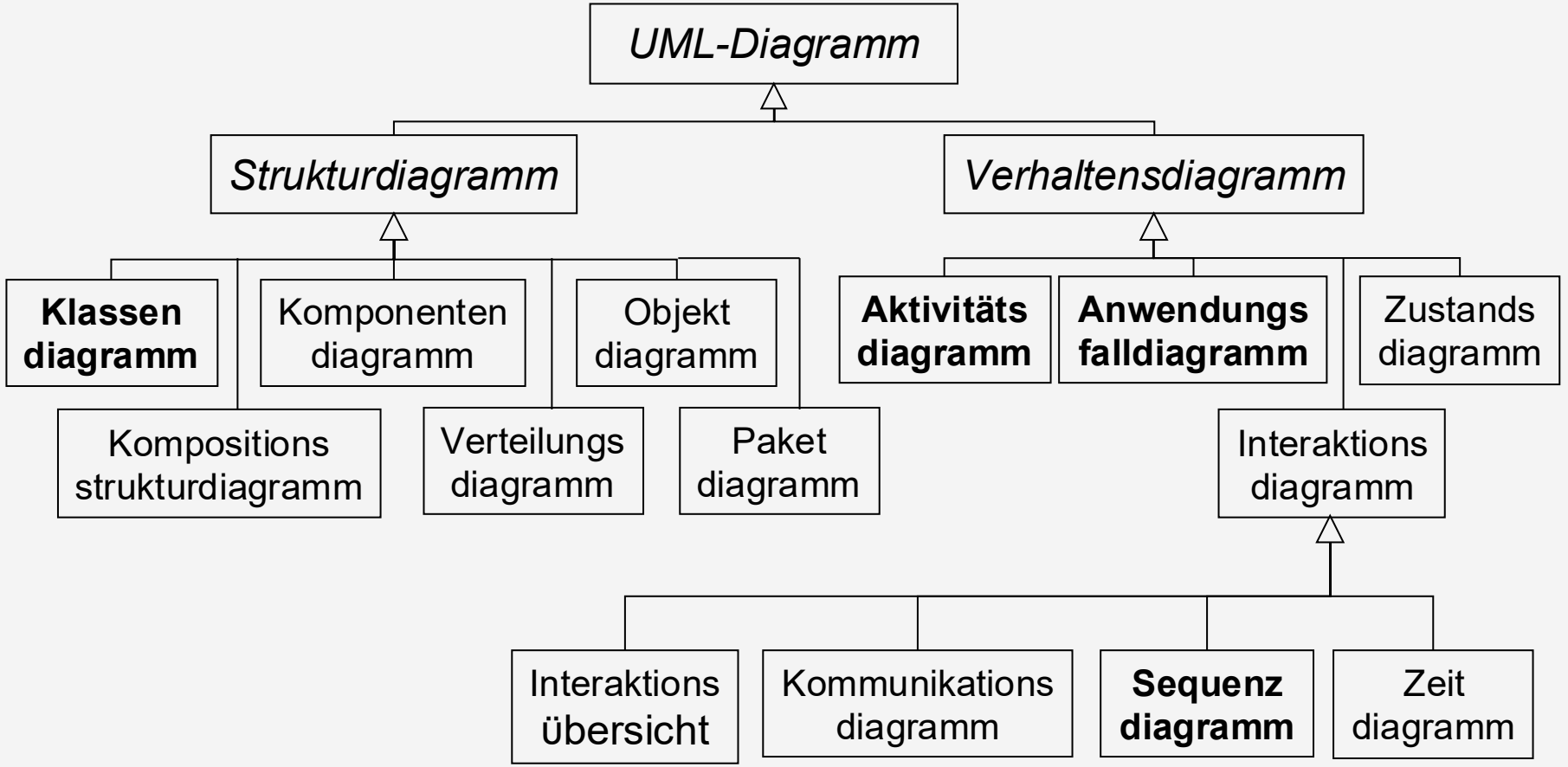
Jede Sprache hat **Stärken und Schwächen.**

Unified Modeling Language

- Graphische Modellierungssprache für Software-Systeme
- Definiert Menge von Diagrammtypen
- Mehrere Diagramme bilden Sicht auf ein UML-Modell



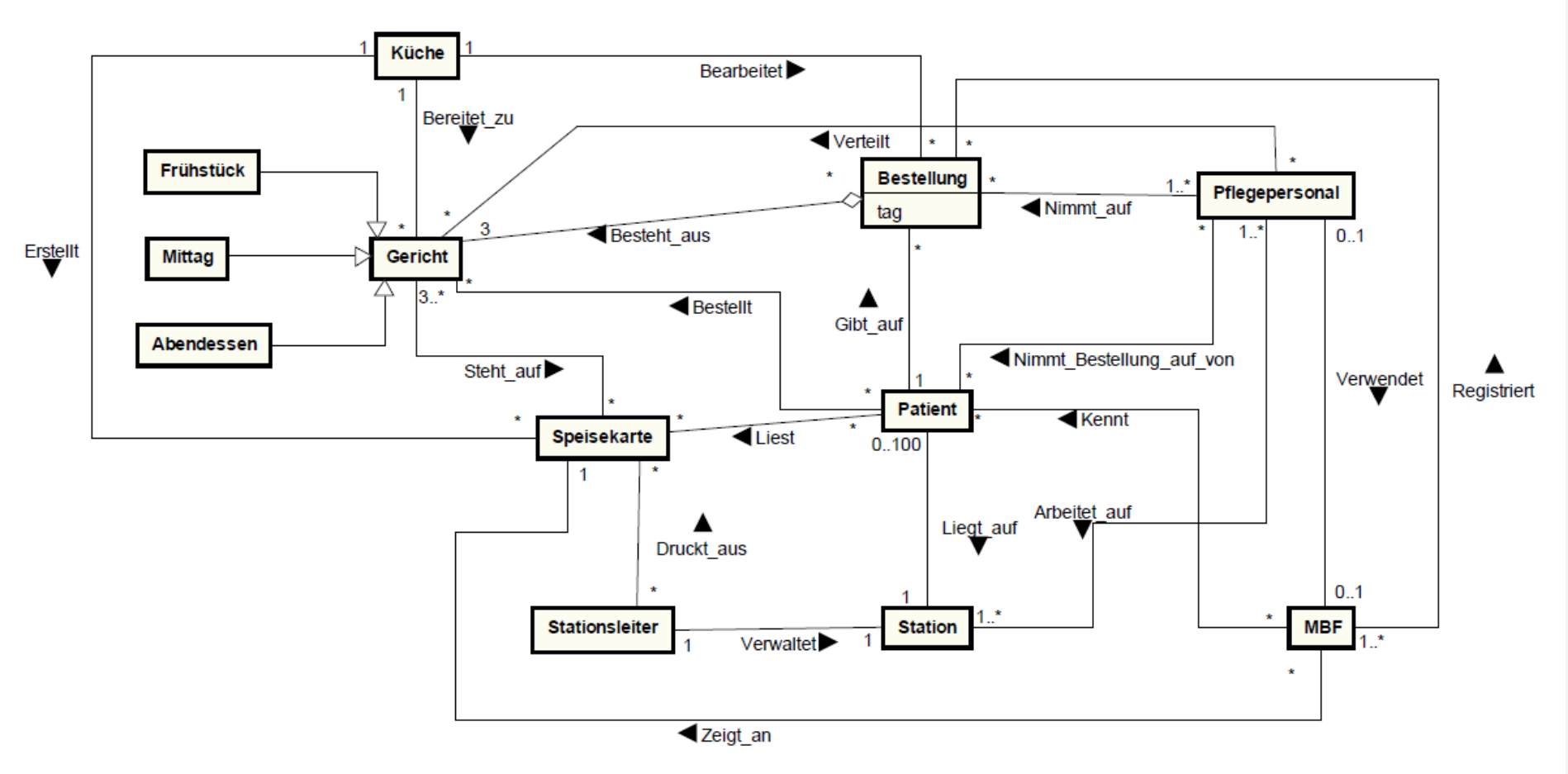
Diagrammtypen



Klassendiagramm

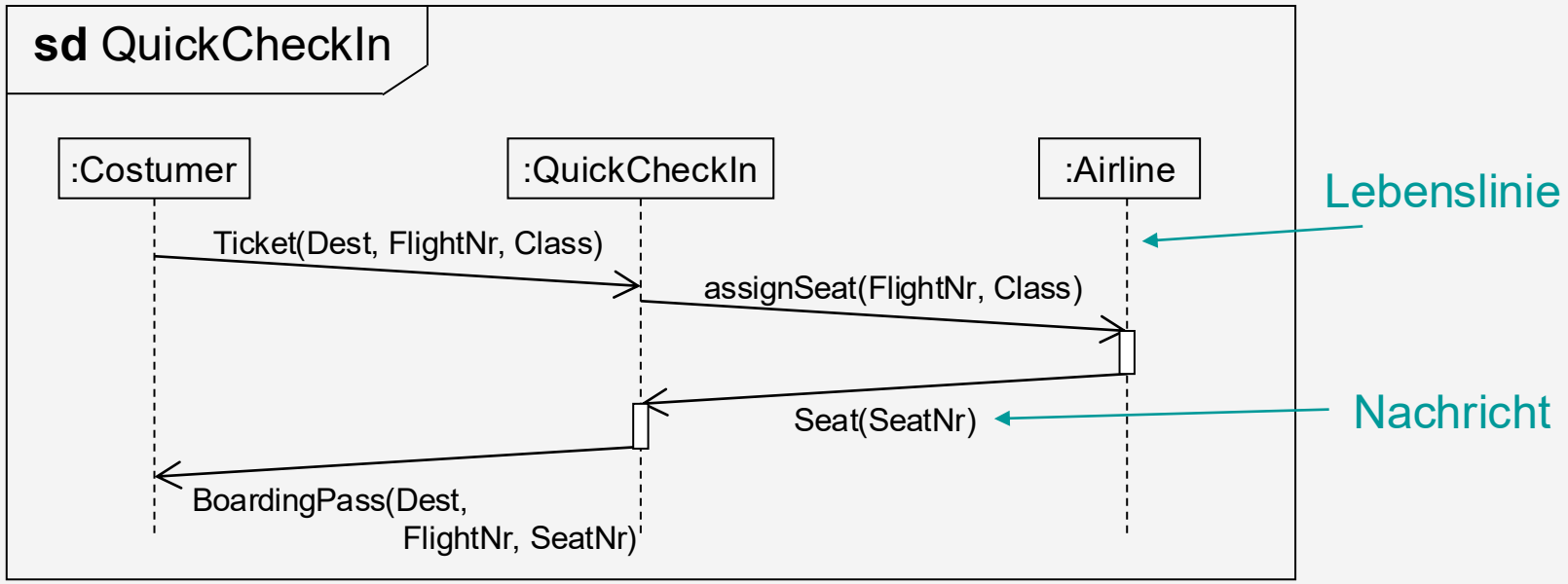
„**Kern**“ eines UML-Modells

- stellt die statische Struktur dar
- Fokus auf (statischen) Objekten



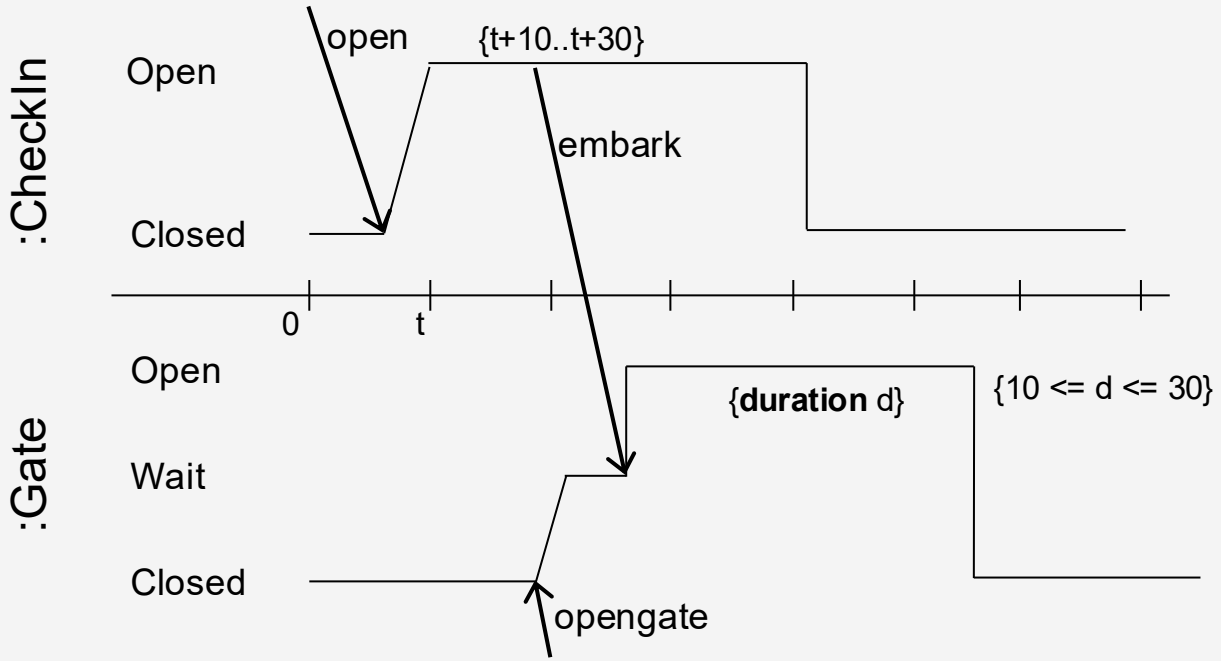
Sequenzdiagramme

- sichtbares Verhalten als Kommunikationssequenz
- exemplarisch auf der Grundlage von konkreten Instanzkonfigurationen
- Darstellung gültiger oder nicht zulässiger Abläufe



Zeitdiagramme

- Zustandswechsel im Zeitverlauf
- Zeit kann sehr präzise modelliert werden



- 10 bis 30 Minuten nach dem Beginn des Eincheckens kann das Einsteigen beginnen
- Die Dauer des Einsteigens soll 60 Minuten nicht übersteigen

Profile

- Unterstützung von Domänen-spezifischen Anforderungen
- Erweitern UML um Mengen von:
  - Stereotypen (Spezialisierung/Einschränkung von Klassen)
  - Tagged Values (Eigenschaften von UML-Elementen)
  - Constraints (z.B. in der Object Constraint Language, OCL)

Profil für eingebettete Echtzeitsysteme

- Tagged Values
  - Adressen, Kapazitäten, Zugriffsarten
  - Ausführungszeiten, Perioden, Anfangs- u. Endzeitpunkte
  - Zeitschranken, Prioritäten, Scheduling-Policy
- Stereotypen
  - Ressourcen, Aktionen und Ereignisse mit assoziierten Tagged Values
- Constraints
  - z.B. Zeitbedingungen
- UML Standard: Modeling and Analysis of Real-Time and Embedded systems (MARTE)

MARTE

- UML Profil für **M**odeling and **A**nalysis of **R**eal-**T**ime and **E**mbedded systems
  - verfügbar als UML2-Profil, aktuell: Version 1.2. (seit 2019)
- Nicht-funktionale Eigenschaften: Notationen zur Definition von Zeit, Energie, …
- Zeitmodell: zur Definition von physikalischen und logischen Zeiteigenschaften
  - CCSL: Clock Constraint Specification Language
- Generische Ressourcenmodellierung: Erweiterungen für Modellierung einer Plattform für Echtzeit- und eingebettete Anwendungen
- Allokationsmodellierung: Zuordnung von Anwendungselementen auf Plattformelemente

UML: Vor- und Nachteile

Diskutieren Sie Vor- und Nachteile von UML für die Modellierung eingebetteter Systeme!

UML: Vor- und Nachteile

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

- ✓ Komponenten können verteilt entwickelt und ausgetauscht werden (*Modularity*)
- ✓ Komponenten können wiederverwendet werden (*Reusability*)
- ✓ Kapselung des internen Aufbaus realisiert das Prinzip des *Information hiding*
- ✓ Die Konzentration auf die Schnittstellen führt zu einem transparenten Entwurf
- ✓ Unterstützung von Zeit und Ressourcen durch spezielle Profile

- Nur teilweise ausführbar
- Fokus auf statischer Struktur, Objekte statt Prozesse
- Informelle Semantik ist nicht verfizierbar

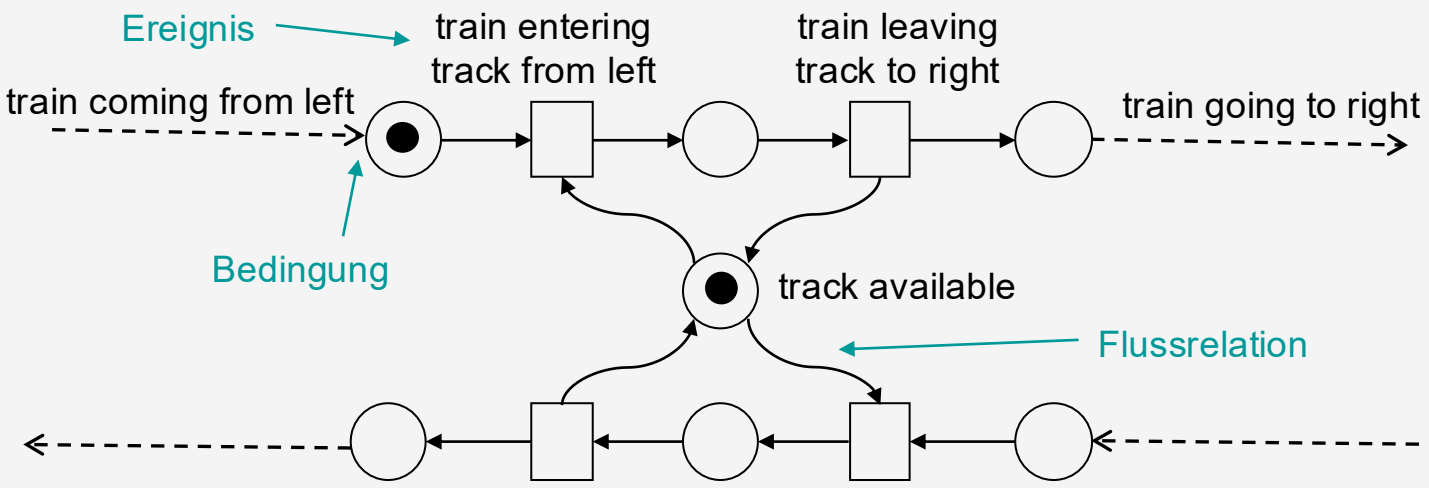
Petri-Netze

- 1962 von Carl Adam Petri vorgestellt
- Modellierung kausaler Zusammenhänge nebenläufiger und verteilter Systeme
- Kommunikation über *message passing*
- kein globaler Synchronisations-/Kommunikationsmechanismus
  - alle Bedingungen werden lokal ausgewertet
- basieren auf Condition/Event Nets

Condition/Event Nets

- Bedingungen (Kreise), Ereignisse (Rechtecke)
- Flussrelationen (Pfeile)
- Höchstens ein Token (Belegung) pro Bedingung

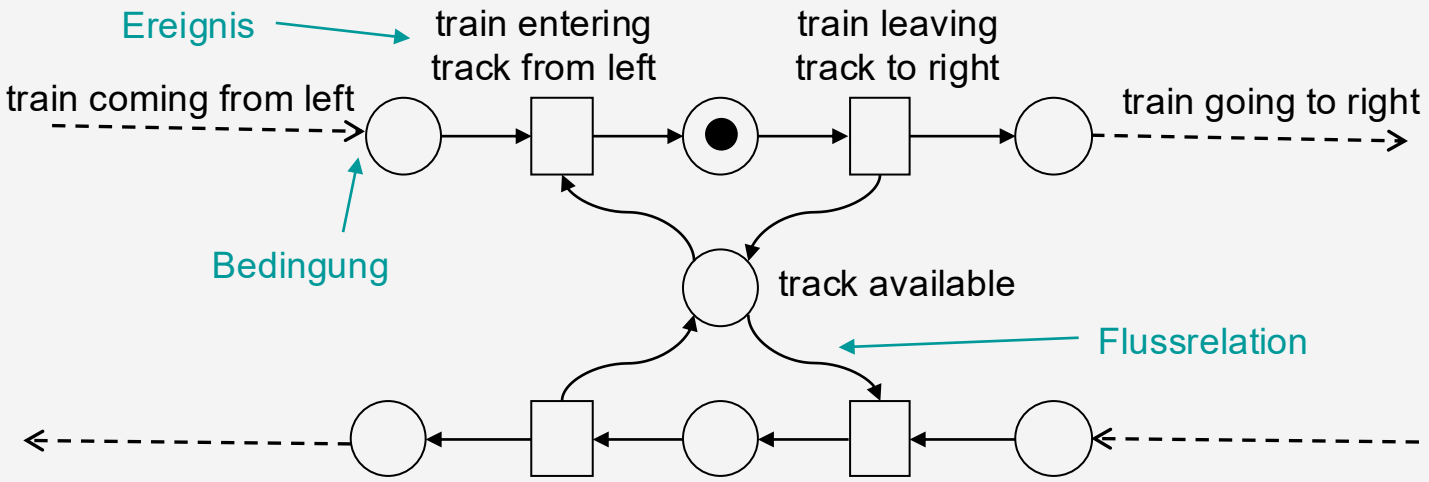
Beispiel: Synchronisation eines eingleisigen Schienenabschnitts



Condition/Event Nets

- Bedingungen (Kreise), Ereignisse (Rechtecke)
- Flussrelationen (Pfeile)
- Höchstens ein Token (Belegung) pro Bedingung

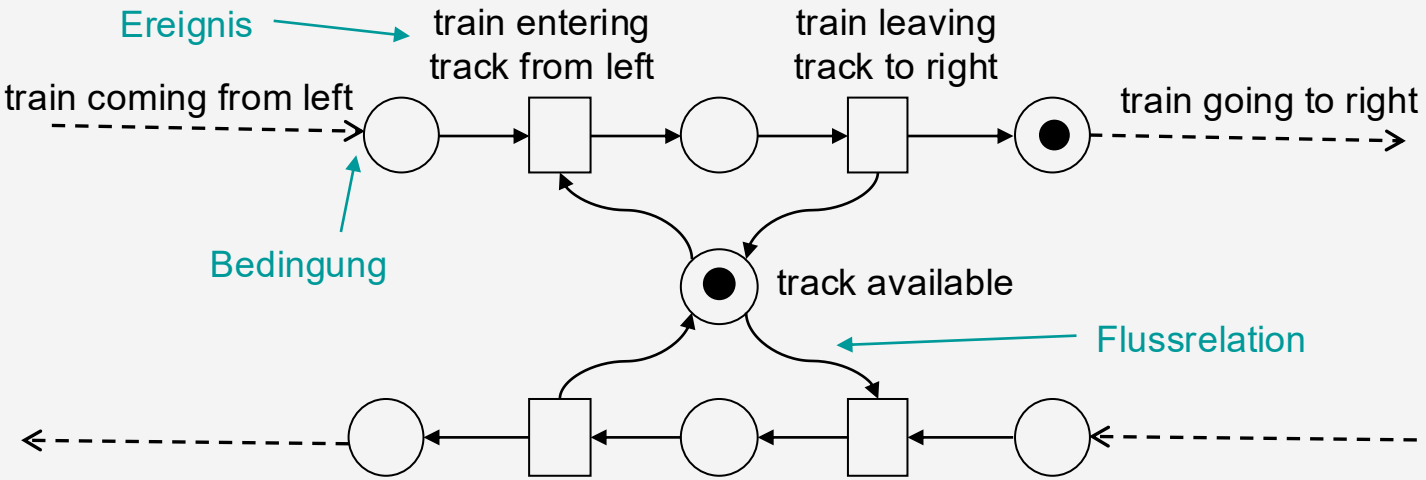
Beispiel: Synchronisation eines eingleisigen Schienenabschnitts



Condition/Event Nets

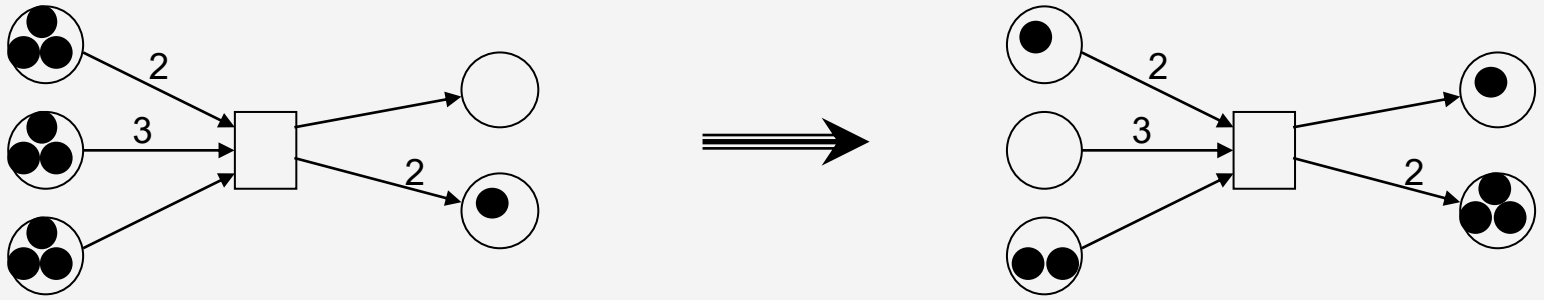
- Bedingungen (Kreise), Ereignisse (Rechtecke)
- Flussrelationen (Pfeile)
- Höchstens ein Token (Belegung) pro Bedingung

Beispiel: Synchronisation eines eingleisigen Schienenabschnitts



Place/Transition Nets

- *Places* oder Stellen (Kreise) und Transitionen (Rechtecke)
- Anzahl der Tokens in den Stellen = Markierung
- Transition zerstört Token an Ausgangsstelle und erschafft an Zielstelle
- Anzahl der Tokens wird als Kantengewicht angegeben
  (vor der Transition: Bedingung, danach: Resultat)



Formale Semantik

Versucht, das mal formal zu modellieren!

Syntax und Semantik Petrinetze

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

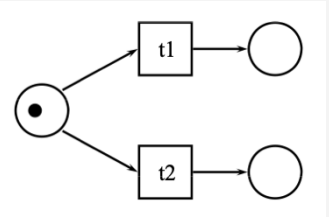
Syntax

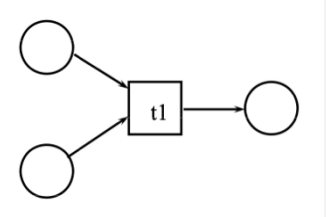
Petrinetz ist Tripel $\mathcal{N} = (\mathrm{S}, \mathrm{T}, \mathrm{F})$ bestehend aus

- S – Places / Stellen (nichtleere Menge, grafisch Kreise)
- T – Transitions (nichtleer, grafisch Rechtecke)
- $\mathrm{S} \cap \mathrm{T} = \emptyset$
- $\mathrm{F} \subseteq (\mathrm{S} \times \mathrm{T}) \cup (\mathrm{T} \times \mathrm{S})$, Flow (Flussrelation – N.B. es wird nie Stelle mit Stelle oder Transition mit Transition direkt verbunden)

Semantik

- Netz wird um Markierungen erweitert: $m: S \to \mathbb{N}$ und Gewichte $\mathrm{w}: (\mathrm{S} \times \mathrm{T}) \cup (\mathrm{T} \times \mathrm{S}) \to \mathbb{N}$ mit $\mathrm{w}(\mathrm{x}, \mathrm{y}) = 0$ falls $(\mathrm{x}, \mathrm{y}) \notin \mathrm{F}$
- Wir verwenden noch
  - Vorbereich eines Elementes x: $\cdot\, x = \{y | (y, x) \in F\}$
  - Nachbereich eines Elementes x: $x \cdot = \{(x, y) \in F\}$
  - Vorwärtsverzweigte Knoten (mit $|x \cdot| > 1$) modellieren Alternativen & Konflikte
  - Rückwärtsverzweigte Knoten (mit $|\cdot\, \mathrm{x}| > 1$) modellieren Synchronisation

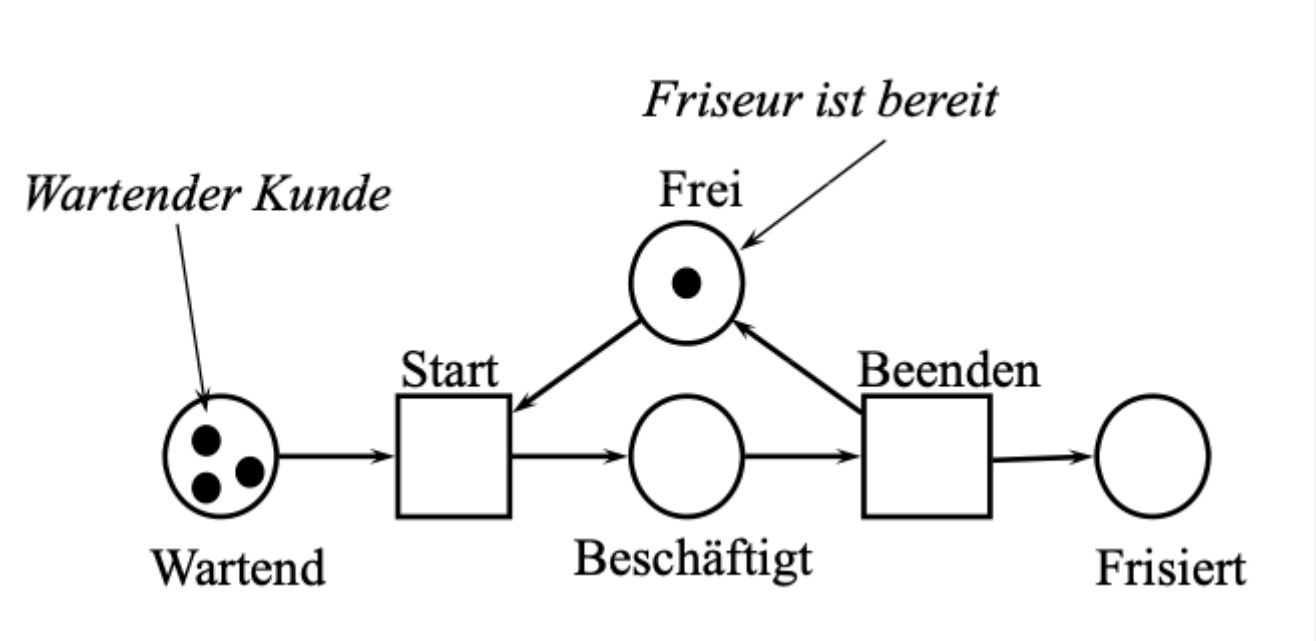




- Eine Markierung aktiviert eine Transition $\mathrm{t} \in \mathrm{T}$, falls $\forall \mathrm{s} \in \cdot\, \mathrm{t}: \mathrm{m}(\mathrm{s}) \geq \mathrm{w}(\mathrm{s}, \mathrm{t})$
- Falls t unter m aktiviert ist, kann t schalten. Das führt zu einer Folgemarkierung m' (in Zeichen $\mathrm{m} \xrightarrow{\mathrm{t}} \mathrm{m}'$) mit

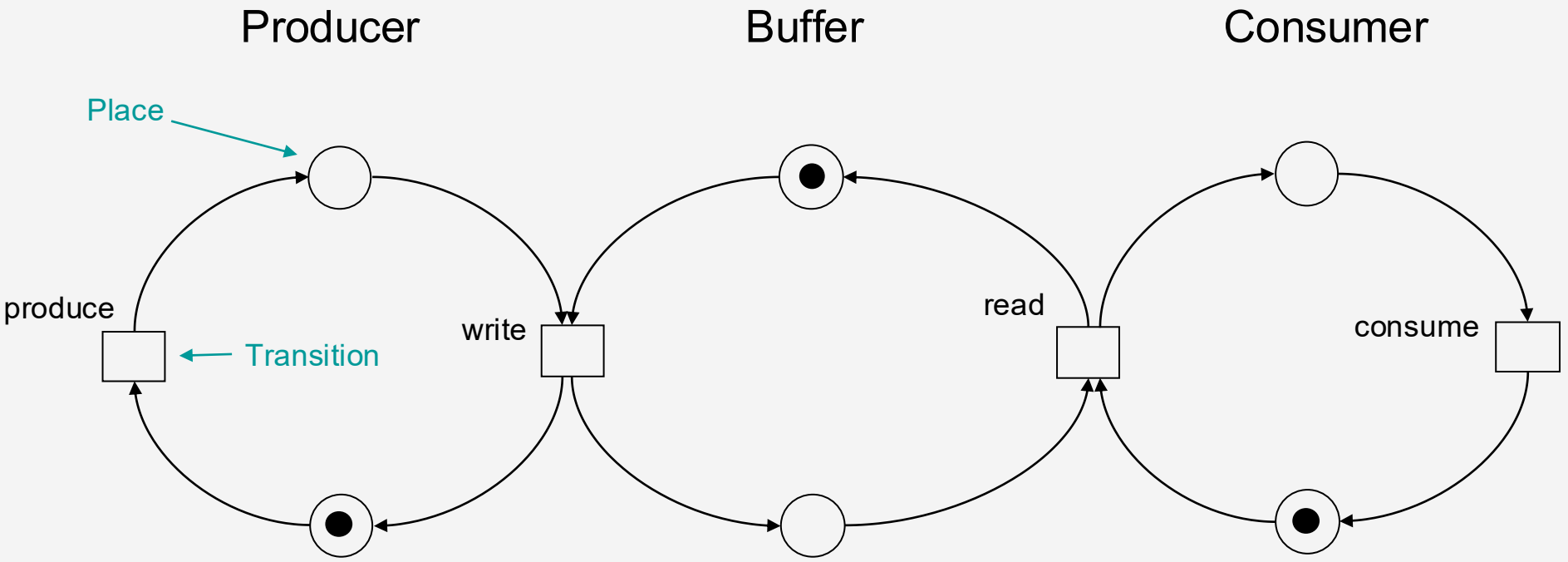
$$\mathrm{m}'(\mathrm{s}) = \mathrm{m}(\mathrm{s}) - \mathrm{w}(\mathrm{s}, \mathrm{t}) + \mathrm{w}(\mathrm{t}, \mathrm{s})$$

Beispiel

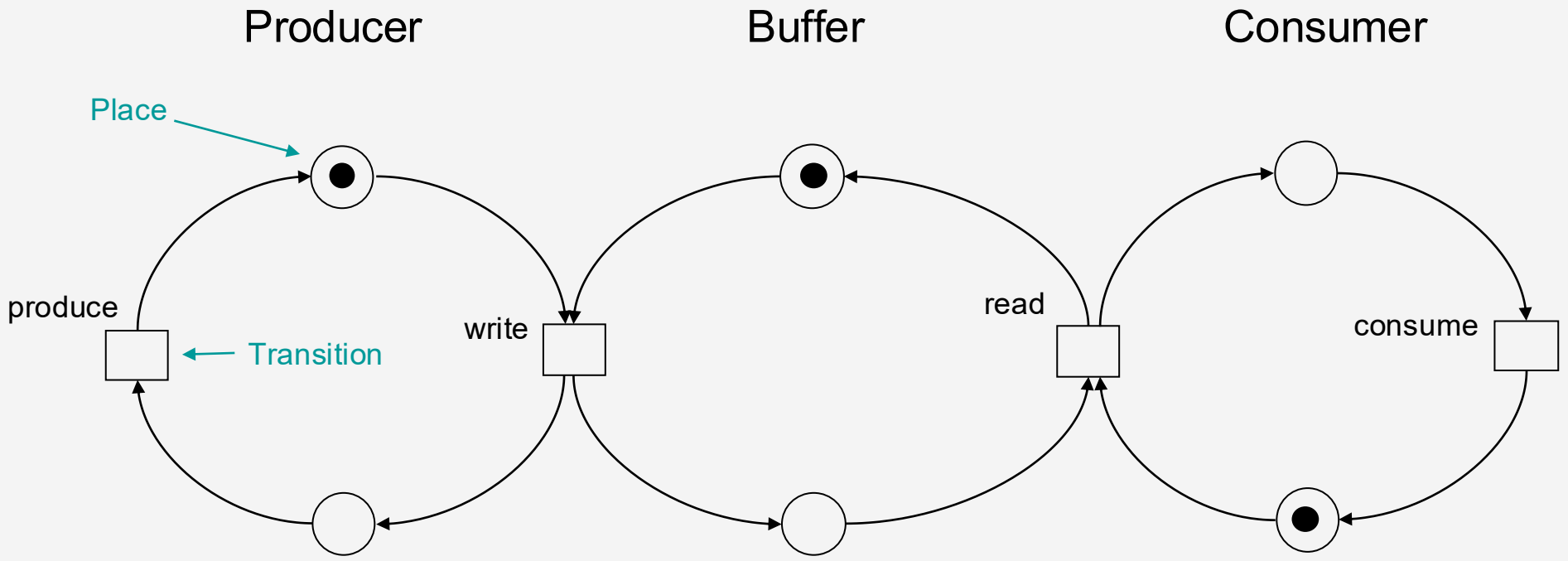


Quelle: https://dbis.ipd.kit.edu/download/kap03-wfms-mod-mit-petrinetzen.pdf

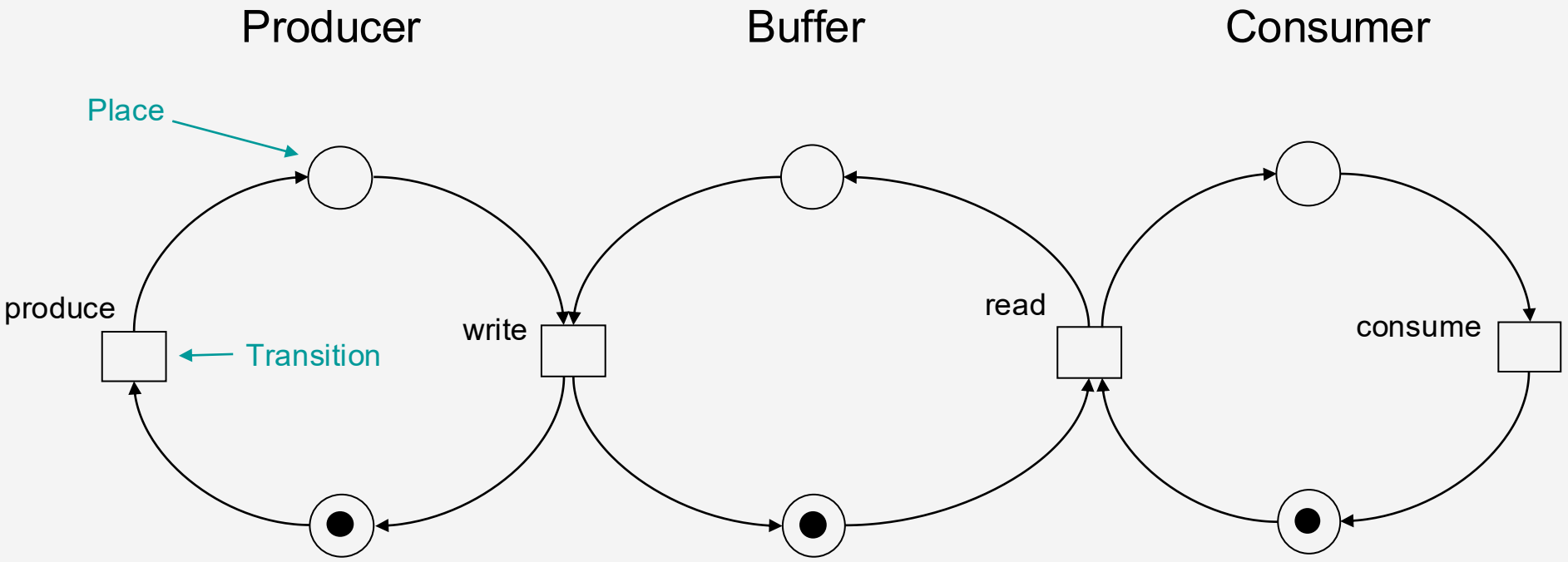
Place/Transition Nets: Beispiel



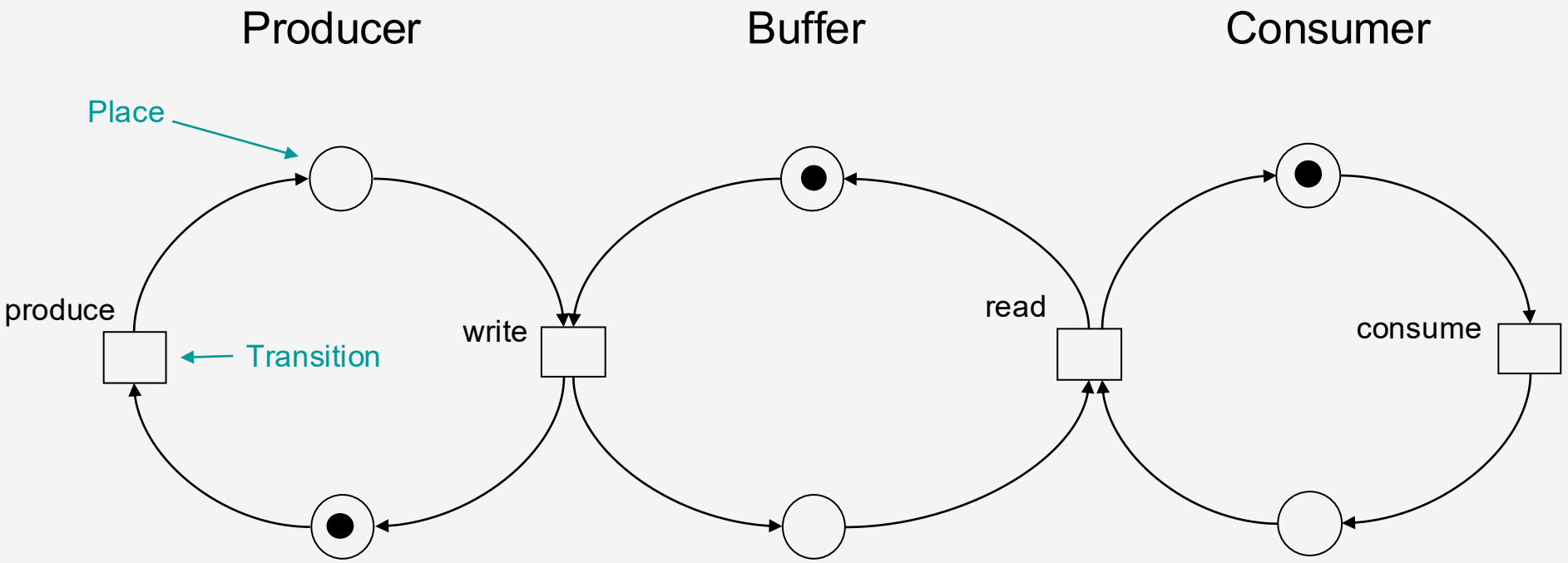
Place/Transition Nets: Beispiel



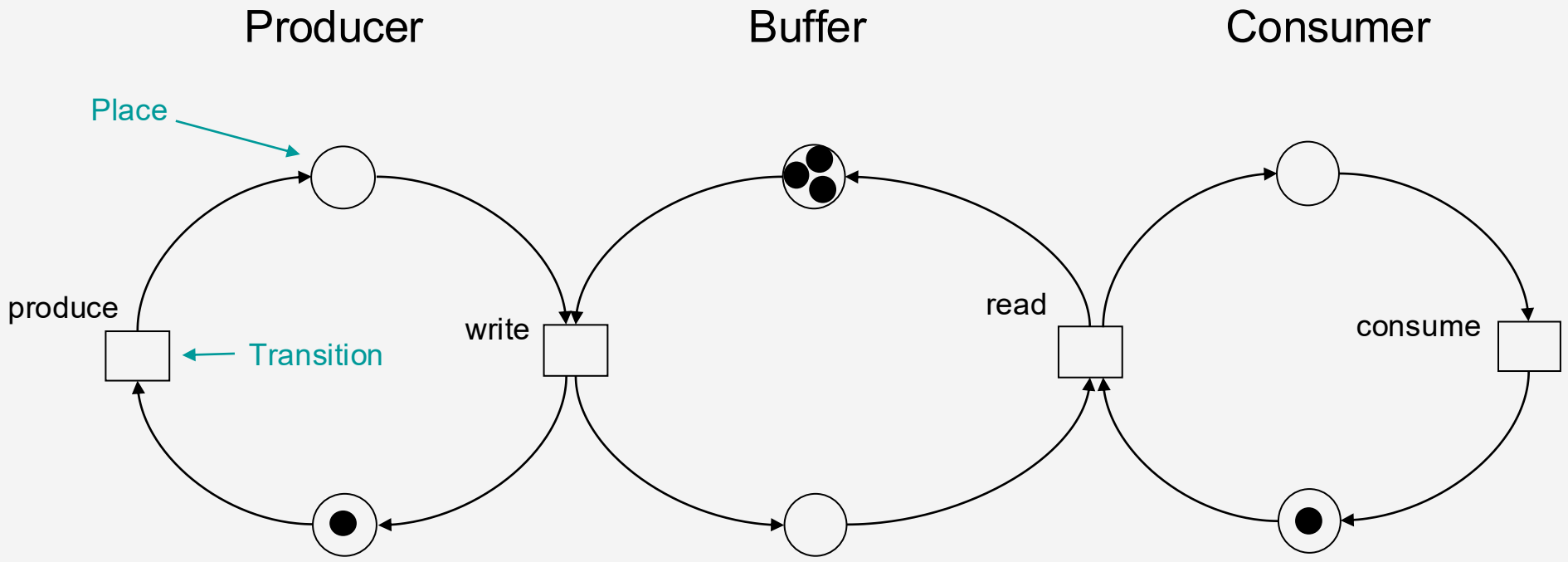
Place/Transition Nets: Beispiel



Place/Transition Nets: Beispiel

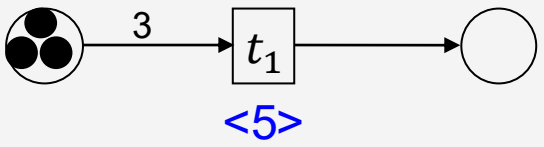


Place/Transition Nets: Beispiel

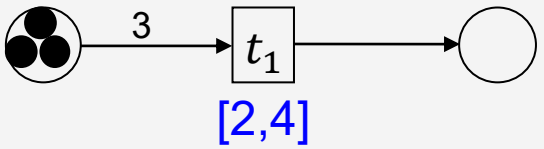


Zeiterweiterungen

- Timed Petri Nets
  - Transition benötigt Zeit d (duration) zum Schalten



- Time Petri Nets
  - Transition darf nur in Zeitintervall $[t_{min}, t_{max}]$ schalten
  - Schalten verbraucht keine Zeit



- es existieren auch Erweiterungen mit zeitbehafteten Stellen

Petri-Netze: Vor- und Nachteile

Diskutieren Sie Vor- und Nachteile von Petri-Netzen für die Modellierung eingebetteter Systeme!

Petri-Netze: Vor- und Nachteile

- ✓ mathematisch präzise Beschreibung
- ✓ Petri-Netze sind simulierbar und formal verifizierbar
- ✓ sehr gut für quantitative Analysen geeignet
- ✓ graphische Darstellung
- ✓ viele Erweiterungen: Zeitbehaftete, stochastische und hierarchiche Petri-Netze
- ✓ Werkzeugunterstützung zur Modellierung, Simulation und Analyse
- ▪ Spezialisiert auf verteilte Systeme
- ▪ globale Kommunikation und Kommunikation zwischen mehreren Komponenten schlecht modellierbar
- ▪ Modularisierung nur beschränkt möglich

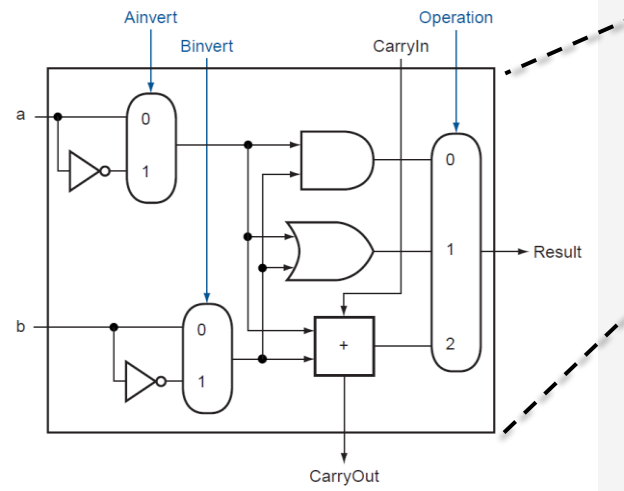
Übersicht über die heutige Vorlesung

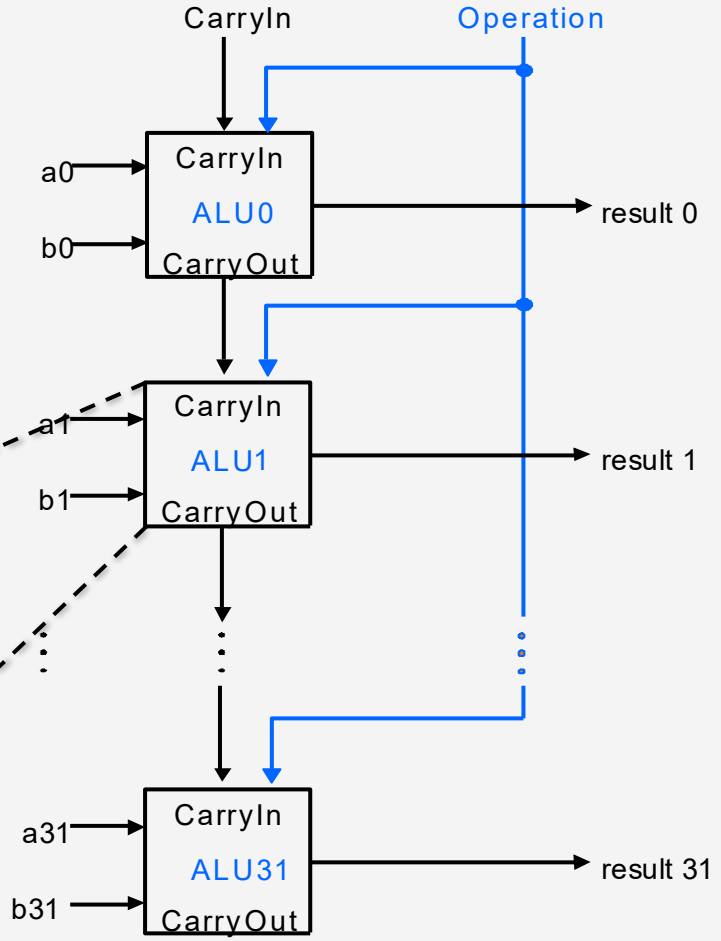
Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

1. Zentrale Anforderungen an Modellierungssprachen
2. Modellierung von Software
   - UML
   - Petri-Netze
3. Modellierung von Hardware
   - Blockdiagramme
   - VHDL
4. Zusammenfassung

Blockdiagramme

- graphische Modellierung digitaler oder dynamischer Systeme
- Blöcke als zentrales Modellierungselement
- Abstraktion des internen Verhaltens
- Verbindung über Linien
- Weit verbreitet im Hardwareentwurf





Hardwarebeschreibungssprachen

- textuelle Beschreibung digitaler Systeme → Implementierung digitaler Schaltungen
- In der Regel auf Register-Transfer-Ebene
- Struktur ähnlich Blockdiagrammen (Entities, Ports, Signalverbindungen)
- Aber: unterstützt Struktur-, Verhaltens- und Datenflussmodellierung
- automatische Synthese von Netzlisten (Logikgatter und deren Verknüpfung)
- Bekannteste Vertreter: VHDL und Verilog
- **VHDL =“Very High speed integrated circuits Description Language”**
- für analoge Systeme zu AHDL (*Altera Hardware Description Language*) erweitert
- *VHDL-AMS* (AMS = analog/mixed signal) integriert mechanische Elemente, Sensoren und Aktoren

VHDL

"Very High Speed Integrated Circuits Description Language"

- **Textuelle** Spezifikation digitaler Systeme
- Quasi-Standard in Europa
- Für analoge Systeme zu AHDL (*Altera Hardware Description Language*) erweitert
- *VHDL-AMS* (AMS = analog/mixed signal) integriert mechanische Elemente, Sensoren und Aktoren (**aktuelle Forschung**)

- **Vorteile**: simulierbar, verifizierbar, synthetisierbar
- **Nachteil**: nur für Hardware-Spezifikation einsetzbar

VHDL: Verhaltensmodellierung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

```
entity NAND is
port(
  A: in std_logic;
  B: in std_logic;
  C: out std_logic
);
end NAND;
```

```
architecture behavioral of NAND is
begin
  process is
  begin
   if a = '1' and b = '1' then
      c <= '0';
   else
      c <= '1';
   end if;
  end process;
end behavioral;
```

VHDL Notation

Black Box / Schnittstelle

Baustein

```
entity ADDER is generic(n: natural
:=2);
port( A: in std_logic_vector(n-1 downto 0); B: in
std_logic_vector(n-1 downto 0);
   carry: out std_logic;
   sum: out std_logic_vector(n-1 downto 0) );
end ADDER;
```

Parameter, 2 ist Standardwert (aber für beliebige Bitbreiten parametrisierbar), Entity und Architektur werden durch Generic parametrisiert

Schnittstelle (Ein/Ausgänge)
n-1 downto 0 definiert Bitbreite (bei default 2 also 2 Bit)

Architektur / Implementierung
Keine Programmausführung, sondern parallele Hardwarebeschreibung

Architekturrumpf

```
architecture behv of ADDER is
signal result: std_logic_vector(n downto 0); begin
result <= ('0' & A)+('0' & B);

   sum <= result(n-1 downto 0); carry <=
   result(n);
end behv;
```

Deklaration, 1 Bit mehr für Übertrag

Anweisungen, die nebenläufig ausgeführt werden, & ist Konkatenation ('0' & A erweitert den Vektor um 1 Bit links)

VHDL: Strukturmodellierung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

```
entity NAND is
port(
  A: in std_logic;
  B: in std_logic;
  C: out std_logic
);
end NAND;
```

```
architecture structural of NAND is
  component NOT_gate
      port(a : in std_logic; b : out std_logic);
  end component;
  component AND_gate
    port(a : in std_logic; b : in std_logic; c : out std_logic);
  end component;
signal and2not: std_logic;
begin
      and1 : AND_gate port map(A,B,and2not);
      not1 : NOT_gate port map(and2not,C);
end structural;
```

VHDL: Datenflussmodellierung


```
entity ADDER is
port(
              a: in std_logic;
              b: in std_logic;
              carry: out std_logic;
              sum: out std_logic
);
end ADDER;
-----------------------------------------
architecture dataflow of ADDER is
begin
sum <= a xor b;
       carry <= a and b;
end behavioral;
```

Missverständnisse Nebenäufigkeit

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Beispiel:

a <= b;

c <= a;

Bedeutet nicht: c bekommt sofort den Wert b, sondern

- a bekommt neuen Wert
- c sieht noch alten Wert von a

Warum?

Missverständnisse Nebenäufigkeit

Beispiel:

a <= b;

c <= a;

Bedeutet nicht: c bekommt sofort den Wert b, sondern

- a bekommt neuen Wert
- c sieht noch alten Wert von a

Warum?

- Alles passiert nebenläufig; Signale sind Leitungen, keine Variablen!

→ MARS Simulation eines Prozessors aus Rechnerarchitektur-Vorlesung

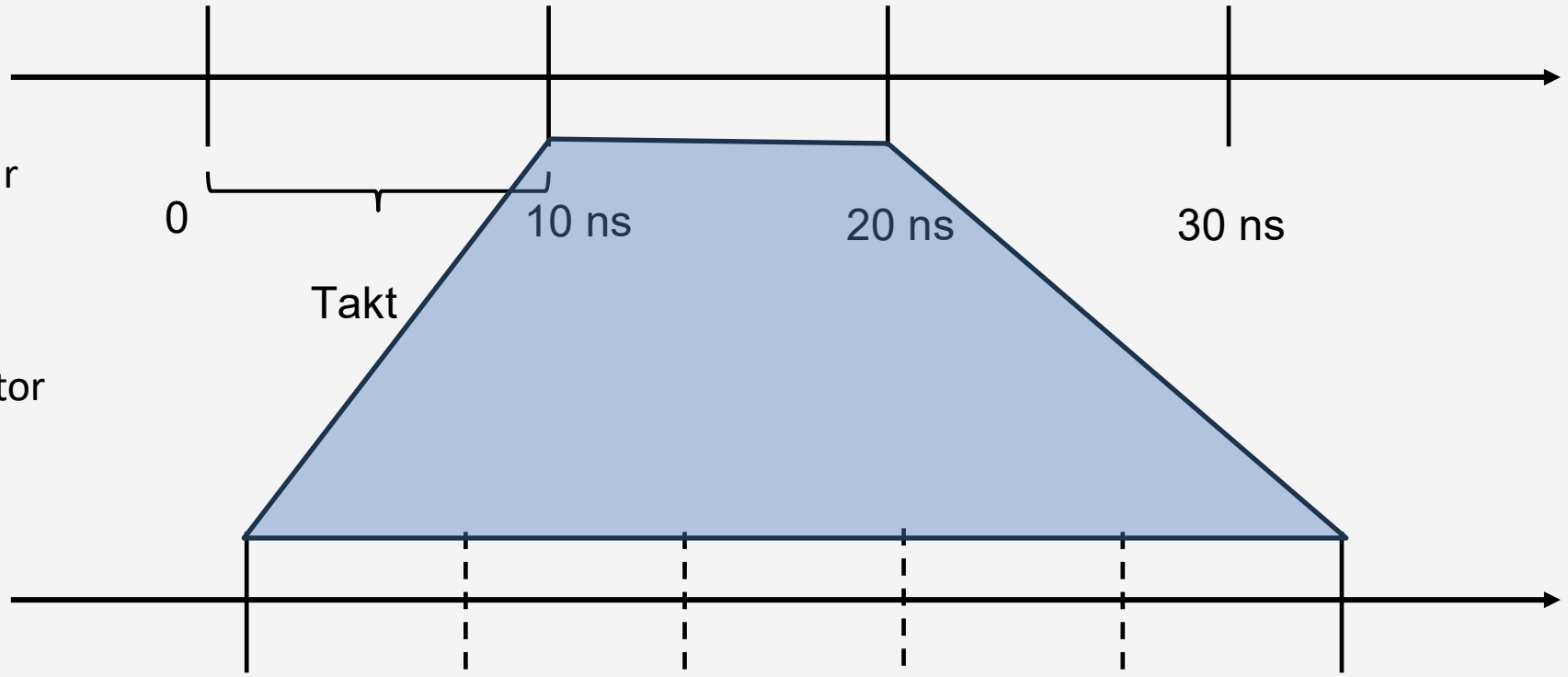
→ Signaländerungen passieren erst nach einem Delta-Zyklus

Was ist ein Delta-Zyklus?

Reale Hardware:

- Logik stabilisiert sich nach der Taktflanke (muss vor dem nächsten Takt stabil sein)
- Delta-Zyklen modellieren Signalpropagation im Simulator

Was zwischendurch passiert und modelliert werden muss:

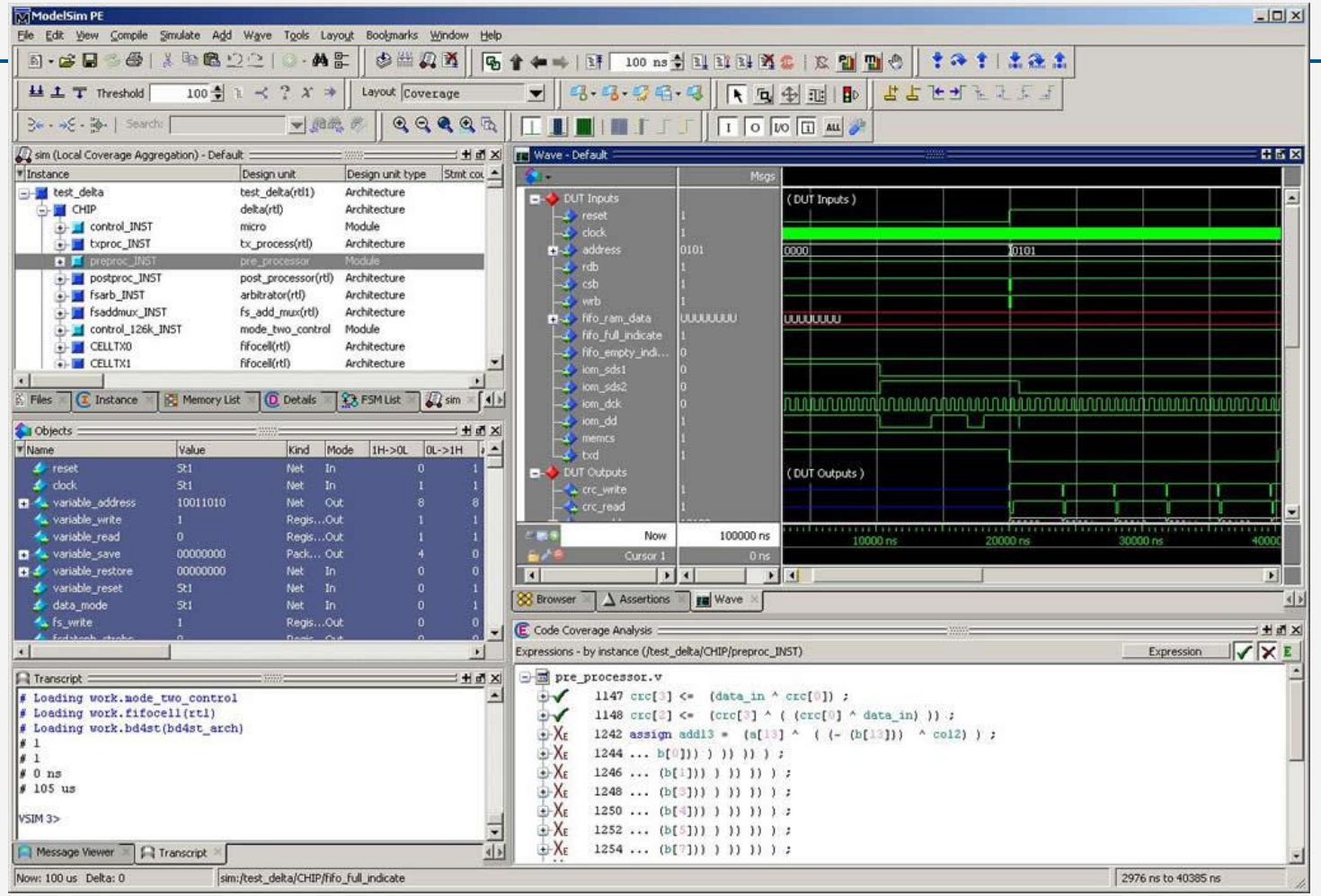


Delta-Zyklen - zwischen Takten stabilisiert sich die kombinatorische Logik nebenläufig

Simulation und Verifikation

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

- VHDL Code kann werkzeuggestützt simuliert und verifiziert werden (z.B. in ModelSIM)
- Verhalten wird in Test-bench eingebettet analysiert
- Verifikation formal oder durch vollständige Simulation



Blockdiagramme und VHDL: Vor- und Nachteile

Diskutieren Sie Vor- und Nachteile von Blockdiagrammen und VHDL für die Modellierung eingebetteter Systeme!

Blockdiagramme: Vor- und Nachteile

- ✓ graphische Modellierung, realisiert *Information Hiding (hierarchische Blöcke kapseln Details, ähnlich Module / Funktionen)*
- ✓ einfache Modellierung von Hardware
- ✓ intuitive Darstellung
- ✓ Werkzeugunterstützung (z.B. ModelSim, Simulink, LabView, Scilab)

- nicht (direkt) ausführbar bzw. Semantikanbhängigkeit vom Tool (z.B. Verilog, VHDL)
- Semantik nicht eindeutig definiert
- Fokus auf Datenfluss, Kontrollfluss oft nur implizit sichtbar

VHDL: Vor- und Nachteile

- ✓ zur Modellierung von Hardware sehr gut geeignet
- ✓ Parallelität und Zeit werden unterstützt
- ✓ Struktur- und Verhaltenssicht
- ✓ ausführbar und verifizierbar (Semantik eindeutig definiert)
- ✓ sehr ausgereifte Werkzeugunterstützung

- keine für Software geeigneten Abstraktionen
- kein effizientes Zielmodell für Softwareausführung
- führt zu Hardwareimplementierung statt Programmausführung

VHDL: Vor- und Nachteile

- ✓ zur Modellierung von Hardware sehr gut geeignet
- ✓ Parallelität und Zeit werden unterstützt
- ✓ Struktur- und Verhaltenssicht
- ✓ ausführbar und verifizierbar (Semantik eindeutig definiert)
- ✓ sehr ausgereifte Werkzeugunterstützung

- keine für Software geeigneten Abstraktionen
- kein effizientes Zielmodell für Softwareausführung
- führt zu Hardwareimplementierung statt Programmausführung

Was unterscheidet Hardware von Software in der Ausführung?

VHDL: Vor- und Nachteile

- ✓ zur Modellierung von Hardware sehr gut geeignet
- ✓ Parallelität und Zeit werden unterstützt
- ✓ Struktur- und Verhaltenssicht
- ✓ ausführbar und verifizierbar (Semantik eindeutig definiert)
- ✓ sehr ausgereifte Werkzeugunterstützung

- keine für Software geeigneten Abstraktionen
- kein effizientes Zielmodell für Softwareausführung
- führt zu Hardwareimplementierung statt Programmausführung

Was unterscheidet Hardware von Software in der Ausführung?

- Software wird sequentiell ausgeführt; Zeitverlauf entsteht durch sequentielle Abarbeitung auf Prozessor
- Hardware arbeitet physisch parallel (Addierer, Register, Multiplexer, Busse etc., alle gleichzeitig); Zeit entsteht durch Signalverzögerungen, Taktsignale, Schaltungen
- In VHDL:
  a <= b and c;
- d <= a or e; bedeutet: gleichzeitige Verarbeitung, nicht sequentiell
- Kein Kontrollfluss, Funktionen, Rekursion, Datenstrukturen etc.

Übersicht über die heutige Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

1. Zentrale Anforderungen an Modellierungssprachen
2. Modellierung von Software
   - UML
   - Petri-Netze
3. Modellierung von Hardware
   - Blockdiagramme
   - VHDL
4. Zusammenfassung

Zusammenfassung

- zentrale Anforderungen an Modellierungssprachen für eingebettete Systeme:
- Nebenläufigkeit
- Kommunikation
- synchrones und asynchrones Verhalten
- Berücksichtigung von Zeit (und ggf. auch Energie)
- Ausführbarkeit (für Simulation und Test)
- Eindeutigkeit / Vorhersagbarkeit / Verifizierbarkeit

Zusammenfassung

UML intuitiv, gut geeignet für komplexe SW-Systeme

informelle Semantik, nicht verifizierbar

Petri-Netze gut geeignet für Nebenläufigkeit

wenig Abstraktion, schlecht für komplexe SW-Systeme

Blockdiagramme gut geeignet für Hardware (synchron/parallel)

Kontrollfluss und Datenstrukturen nicht explizit sichtbar

VHDL “quasi Standard” in Europa für Hardware Design

simulierbar, verifizierbar, synthetisierbar

aber: keine Abstraktionen für Software